SCRUM PROJECT: 3D HOUSE FLOOR PLAN DESIGNER

MINI PROJECT REPORT

Submitted by

BASIL ELDHO
(SNG25MCA-2029)

GINO PETER
(SNG25MCA-2036)

JESIL C J
(SNG25MCA-2040)

Under the guidance of

Ms. ROHINI RAJU

Asst.Professor

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree

of

Master of Computer Application



Department of Computer Applications
Sree Narayana Gurukulam College of Engineering (SNGCE),
Kadayiruppu

November 2026

DECLARATION

I undersigned hereby declare that the mini project report "**3D HOUSE FLOOR PLAN DESIGNER**", submitted for partial fulfilment of the requirements for the award of degree of Master of Technology of the APJ Abdul Kalam Technological University, Kerala is a Bonafide work done by me under supervision of Asst.Professor. **Ms. ROHINI RAJU**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Kadayirippu Signature:

Date: Name of the student:

DEPARTMENT OF COMPUTER APPLICATIONS

SREE NARAYANA GURUKULAM COLLEGE OF ENGINEERING (SNGCE)

KADAYIRUPPU.

CERTIFICATE

This is to certify that the report entitled **"3D HOUSE FLOOR PLAN DESIGNER"** submitted by **BASIL ELDHO (SNG25MCA-2029), GINO PETER (SNG25MCA-2036), JESIL C J (SNG25MCA-2040)** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications is a Bonafide record of the project work carried out by him under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Internal Supervisor

Project Coordinator

HEAD OF THE DEPT

CONTENTS

NO	TITLE	PAGE
1	ACKNOWLEDGEMENT	i
2	ABSTRACT	ii
3	LIST OF TABLES	iii
4	LIST OF FIGURES	iv
5	ABBREVIATIONS	v
6	NOTATION	vi
7	SCRUM PROJECT	vii
8	TEAM ROLES AND RESPONSIBILITIES	x
9	VISION AND MISSION	xii
10	CHAPTER 1. PROJECT IDENTITY	
	1.1 Problem Story	1
	1.1.1 Problem Statement	1
	1.1.2 Relevance to Society / Industry	2
	1.1.3 Requirements Survey	2
	1.2 Project Overview – Proposed Solution	4
	1.2.1 Solution Concept	4
	1.2.2 Key Features	5
	1.3 Project Analysis – Existing / Similar Vs. Proposed	7
	1.3.1 Existing / Similar Approaches	7
	1.3.2 Proposed Uniqueness / Improvement	8
	1.4 Use Cases	9
11	CHAPTER 2. PRODUCT BACKLOGS	
	2.1 Outline	12
	2.2 Integrated Initial Record	13
	2.3 Use Case UC-01 – Open Design Interface	14
	2.4 Use Case UC-02 – Create A Floor Plan	15
	2.5 Use Case UC-03 – ADD AND EDIT WALLS	16

	2.6 Use Case UC-04 – PLACE DOORS AND WINDOWS	17
	2.7 Use Case UC-05 – ARRANGE FURNITURE AND INTERIOR ELEMENTS	18
	2.8 Use Case UC-06 – GENERATE A 3D REPRESENTATION	18
	2.9 Use Case UC-07 – INTERACT WITH THE 3D VIEW	19
	2.10 Use Case UC-08 – MODIFY AN EXISTING FLOOR PLAN	20
	2.11 Use Case UC-09 – NAVIGATE THE DESIGN INTERFACE	21
	2.12 Use Case Uc-10 – USE A RESPONSIVE BROWSER-BASED INTERFACE	22
	2.13 Use Case Uc-11 — Save And Load Floor Plans	22
	2.14 Use Case Uc-12 — Optimize 3d Rendering and Loading Performance	23
12	CHAPTER 3. STORY POINT ESTIMATION	
	3.1 Introduction to Story Point Estimation	26
	3.2 OBJECTIVES OF STORY POINT ESTIMATION	26
	3.3 ESTIMATION APPROACH AND SCALE	26
	3.4 USER STORY ESTIMATION REGISTER	27
	3.5 RATIONALE FOR STORY POINT ASSIGNMENT	28
	3.6 STORY POINT DISTRIBUTION ANALYSIS	28
	3.7 TEAM-WISE WORKLOAD DISTRIBUTION	29
	3.8 ESTIMATED EFFORT AND STORY POINT RELATIONSHIP	30
13	CHAPTER 4. RELEASE & SPRINT PLAN	
	4.1 Introduction	31
	4.2 Product Backlog Prioritization and Estimation	31
	4.3 PLANNED DETAILED SCHEDULE	32
	4.4 DESIGN ASPECTS FOR PRIORITIZED REQUIREMENTS	33
	4.5 ARCHITECTURAL OVERVIEW	34
	4.6 LOGICAL DATA MODEL / ER CONCEPT	34
	4.7 SIX-SPRINT PLAN	35
	4.8 UI/UX WIREFRAME PLAN	35

	4.9 TECHNOLOGY STACK	36
14	CHAPTER 5. PROJECT PROGRESS AND MANAGEMENT	
	5.1 Sprint 1 – Requirements Analysis and Project Setup	37
	5.2 Sprint 2 – 2D FLOOR PLAN EDITOR	40
	5.3 Sprint 3 – 3D VISUALIZATION AND INTERIOR DESIGN	43
	5.4 Sprint 4 – PROJECT STORAGE, SHARING AND BLUEPRINT IMPORT	45
	5.5 Sprint 5 – ENGINEER WORKFLOWS, EXPORT AND COLLABORATION	47
	5.6 Sprint 6 – FINAL TESTING, DEPLOYMENT AND DOCUMENTATION	49
15	CHAPTER 6. BURN DOWN MONITORING	
	6.1 Introduction to Burndown Monitoring	53
	6.2 Sprint Burndown	53
	6.3 SPRINT-WISE BURNDOWN PLAN	54
	6.4 PRODUCT BURNDOWN	56
	6.5 BURNDOWN MONITORING AND CORRECTIVE ACTIONS	58
16	CHAPTER 7. SCRUM CEREMONY (DAILY SCRUM CONSOLIDATION)	
	7.1 INTRODUCTION	60
	7.2 SCRUM CEREMONY FRAMEWORK	60
	7.3 SPRINT GOALS AND DAILY SCRUM CONSOLIDATION	61
	7.4 DAILY SCRUM CONSOLIDATION EVIDENCE REGISTER	61
	7.5 IMPEDIMENT AND ACTION REGISTER	65
	7.6 EVIDENCE REGISTER AND TRACEABILITY	66
17	CHAPTER 8. GITHUB / VERSION CONTROL	
	8.1 VERSION CONTROL AND GIT	68
	8.2 GITHUB REPOSITORY AND DEVELOPMENT WORKFLOW	69
18	CHAPTER 9. TESTING & ACCEPTANCE	
	9.1 Testing Documentation	70
	9.2 TEST PLANS DERIVED	71
	9.3 RESULTS FROM TESTING	74
	9.4 USER ACCEPTANCE TESTING	76

19	CHAPTER 10. FINAL PRODUCT EVIDENCE	
	10.1 Introduction	77
	10.2 USER INTERFACE EVIDENCE	77
	10.3 2D FLOOR PLAN DESIGN EVIDENCE	77
	10.4 3D VISUALIZATION EVIDENCE	77
	10.5 PROJECT MANAGEMENT EVIDENCE	78
	10.6 IMPORT AND EXPORT EVIDENCE	78
	10.7 TESTING AND FUNCTIONALITY EVIDENCE	78
	10.8 FINAL PRODUCT VERIFICATION	79
20	APPENDICES	
	Appendix A –Sample Test Cases	80
	Appendix B – Additional Screenshots	83
21	REFERENCES	91

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our academic mini project, **"3D HOUSE FLOOR PLAN DESIGNER"** undertaken as part of our curriculum at SNGCE, affiliated with Kerala Technological University (KTU).

First and foremost, we extend our sincere thanks to **Prof. Dr. S Jose**, Principal of SNGCE, for fostering an environment that encourages innovation, research, and learning. His commitment to academic excellence has provided us with the resources and opportunities necessary for our academic and technical growth

We are also deeply grateful to **Dr. Sandhya R**, Head of the Department of Computer Applications, for her continuous support, encouragement, and guidance throughout the project. Her leadership and vision have inspired us to strive for excellence in our academic endeavours.

Our special thanks go to our project guide, **Dr. Anoopkumar M**, Associate Professor, whose expertise, mentorship, and constructive feedback have been invaluable throughout the development of this project. His guidance has helped us understand the concepts of blockchain technology, smart contract security, vulnerability analysis, and software development, enabling us to refine our approach and work towards achieving our project objectives.

Our special thanks go to our project guide, **Ms. Rohini Raju**, whose expertise and mentorship were invaluable throughout the development process. Their insights and constructive feedback have helped us refine our approach and achieve our project objectives.

We would like to acknowledge the contributions of other faculty members, who provided us with essential knowledge and support during various stages of our project. Their dedication to teaching has significantly enriched our learning experience.

We are profoundly thankful to **God Almighty** for granting us the strength, wisdom, and perseverance needed to complete this project. We believe that faith has played a crucial role in guiding us through challenges. Lastly, we wish to express our heartfelt appreciation to our parents for their unconditional love, encouragement, and support throughout our academic journey. Their belief in us has been a constant source of motivation This project has not only enhanced our technical skills but also taught us valuable lessons in teamwork and project management. We are excited to carry these experiences forward into our future endeavou

ABSTRACT

The 3D House Floor Plan Designer is a web-based application designed to simplify house planning by enabling users to create, edit, and visualize house floor plans in an interactive three-dimensional environment. Traditional floor-planning methods mainly rely on two-dimensional drawings, which can make it difficult for non-technical users to understand room dimensions, wall heights, furniture placement, and the overall appearance of a proposed house. The proposed system addresses these challenges by integrating a user-friendly 2D floor-plan editor with real-time 3D visualization.

The application allows users to create house projects, define plot dimensions, draw walls, arrange rooms, and add doors, windows, stairs, furniture, and other interior elements. The system converts the designed 2D layout into a three-dimensional house model with realistic wall height and thickness. Users can interact with the generated model through perspective, top-view, and 360-degree viewing modes to examine the layout from different angles. The platform also provides facilities for calculating room dimensions and areas, saving and editing projects, sharing designs with others, and exporting floor plans and project reports

.

The proposed system is developed using modern web technologies, including React.js for the frontend, Three.js and WebGL for 3D visualization, Node.js and Express.js for backend services, and MongoDB for data storage. JSON Web Token (JWT) is used for authentication and access management. The system follows a modular architecture to support project management, floor-plan editing, 3D models generation, furniture placement, and design sharing.

The primary objective of this project is to make residential space planning more accessible, interactive, and understandable for homeowners, students, interior designers, architects, and builders. By combining 2D design capabilities with interactive 3D visualization, the application helps users evaluate their proposed layouts before construction or interior implementation. This project also demonstrates the practical application of full-stack web development, database management, 3D graphics, and Agile Scrum methodologies in developing a real-world software solution.

Keywords: 3D House Floor Plan Designer, 2D-to-3D Conversion, Three.js, React.js, Web-Based Design, Interactive Visualization, House Planning, MongoDB.

LIST OF TABLES

No.	Title	Page No.
1	Story Point Estimation	27
2	Prioritized Product Backlog	31
3	Sprint-Wise Implementation Schedule	32
4	Detailed Sprint Schedule	32
5	Design Mapping for Prioritized Requirements	33
6	Populated Scrum Board	37

LIST OF FIGURES

No.	Title	Page No.
1	Use Case Diagram	9
2	Activity Flow Diagram	11
3	Burndown Chart	57
4	Appendix	80

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
JWT	JSON Web Token
MCA	Master of Computer Applications
MVP	Minimum Viable Product
QA	Quality Assurance
REST	Representational State Transfer
SP	Story Point
UI	User Interface
UX	User Experience
VCS	Version Control System
3D	Three-dimensional
2D	Two-dimensional

NOTATION

Notation	Description
SP	Relative effort estimate expressed in story points.
P0 / High	Essential functionality for a minimum viable product.
P1 / Medium	Important functionality after core editor flow.
P2 / Low	Enhancement or optional improvement.
DoR	Definition of Ready: conditions for a story to enter a sprint.
DoD	Definition of Done: conditions for a story to be accepted as complete.
TBD	To be determined from team records.

SCRUM PROJECT

Introduction to Scrum

Scrum is an Agile framework used to manage complex projects, particularly in software development. It focuses on completing work through short, time-bound iterations called sprints. Scrum encourages teamwork, accountability, transparency, and continuous improvement throughout the project. It helps development teams divide a large project into smaller, manageable tasks, track their progress, and deliver working project outcomes based on the most important requirements. By following this approach, teams can adapt to changes, identify problems early, and improve the quality of their work. **Overview of Academic Mini Projects Using Scrum at APJ Abdul Kalam Technological University (KTU)**

Project Structure

At APJ Abdul Kalam Technological University (KTU), students generally work in small teams to complete academic mini projects using the Scrum framework. Each team selects a specific problem or needs related to its area of study and develops a suitable solution. These projects may involve software development, research activities, or other practical applications of theoretical knowledge. The Scrum framework helps students organize their work, coordinate with team members, monitor progress, and complete the project systematically.

Roles and Responsibilities

In Scrum-based academic mini projects, each team member has specific roles and responsibilities that contribute to the successful completion of the project.

- Product Owner: If there is a customer for the project then he/she will be the Product Owner (External Guide/ HoD) and a faculty from the department will be the Internal Guide. If there is no such customer then the Internal Guide himself/herself shall act as the Product Owner. This role is usually filled by those who represents the stakeholders' interests, prioritizes the project backlog, and defines the project vision.
- Scrum Master: A faculty / technical staff shall act as the Scrum Master to continuously monitor the project development. Periodic meetings, of less than 15 minutes, at the convenience of the Scrum Master are to be highly encouraged. SM facilitates Scrum practices, helps resolve impediments, and ensures the team adheres to Agile principles.
- Development Team: Comprised of students with diverse skill sets, this group collaborates to deliver project increments during each sprint.

Execution Process

The Scrum-based mini project is carried out through a series of iterative cycles called sprints, which typically last two to four weeks. At the beginning of each sprint, the team conducts a sprint planning session to select the user stories or requirements from the project backlog that can be completed during that sprint.

During the development process, short stand-up meetings help team members communicate their progress, share updates, identify challenges, and discuss any difficulties they face. These meetings improve coordination and help the team resolve issues at an early stage.

At the end of each sprint, a sprint review meeting is conducted to demonstrate the completed work to the guide or relevant stakeholders. Feedback is collected during the review and used to identify improvements and update future work priorities. This iterative process continues until the project objectives are achieved. It enables students to develop the project gradually while adapting to feedback and changing requirements.

Learning Outcomes

By using Scrum for academic mini projects, students at KTU gain practical experience in Agile project management and learn how to apply Scrum principles in real-world situations. They develop teamwork, communication, collaboration, and problem-solving skills while learning to manage time, resources, and project activities effectively.

The Scrum approach also helps students understand how to divide complex tasks into smaller activities, prioritize requirements, monitor progress, and improve their work through regular feedback. This hands-on experience prepares students for professional careers, particularly in software development and project management, while strengthening their understanding of project planning, execution, and continuous improvement.

Course Outcomes

After completing the course, students will be able to:

CO No.	Course Outcome (CO)	Bloom's Category Level
CO 1	Identify a real-life project that is useful to society/industry	Level 2: Understand
CO 2	Interact with people to identify the project requirements	Level 3: Apply
CO 3	Apply suitable development methodology for the development of the product/project	Level 3: Apply
CO 4	Analyse and design a software product/project	Level 4: Analyze
CO 5	Test the modules at various stages of project development	Level 5: Evaluate
CO 6	Build and integrate different software modules	Level 6: Create
CO 7	Document and deploy the product/project	Level 3: Apply

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8
CO 1	**2**	**3**	**3**	**3**	**1**	**2**	**3**	**3**
CO 2	**2**	**3**	**2**	**3**	**2**	**3**	**2**	**1**
CO 3	**3**	**3**	**3**	**3**	**3**	**1**	**3**	**3**
CO 4	**3**	**3**	**3**	**3**	**3**	**3**	**3**	**1**
CO 5	**3**	**3**	**3**	**3**	**3**	**2**	**3**	**1**
CO 6	**3**	**3**	**3**	**3**	**3**	**2**	**3**	**3**
CO 7	**1**	**1**	**3**	**3**	**3**	**2**	**3**	**3**

3/2/1: High/Medium/Low

TEAM ROLES AND RESPONSIBILITIES

In an academic project utilizing the Scrum framework, the roles and responsibilities of the team members are crucial for ensuring effective collaboration and successful project outcomes. The Scrum team is composed of three primary roles**: Product Owner, Scrum Master,** and **Development Team.**

Product Owner

The Product Owner acts as the voice of the stakeholders and is responsible for defining the vision of the project. This role involves managing and prioritizing the product backlog, which is a dynamic list of features, enhancements, and fixes that need to be addressed. The Product Owner ensures that the team focuses on delivering maximum value by clearly articulating user stories and acceptance criteria, thus guiding the Development Team on what needs to be built next.

Scrum Master

The Scrum Master serves as a facilitator and coach for the Scrum team. Their primary responsibilities include ensuring that Scrum practices are followed, helping to remove any impediments that may hinder the team's progress, and fostering an environment conducive to collaboration and continuous improvement. The Scrum Master also protects the team from external distractions and encourages open communication among team members, ensuring that everyone understands their roles within the Scrum process.

Development Team

The Development Team consists of cross-functional members who are responsible for delivering potentially shippable increments of the product at the end of each sprint. This team is self-organizing, meaning they have the autonomy to decide how best to accomplish their work. Members typically include software developers, designers, testers, and other specialists who collaborate closely to meet sprint goals. The Development Team is accountable for maintaining high-quality standards and adhering to the "Definition of Done" for each increment.

List of Team Members

- **Product Owner: Ms. Rohini Raju**
- **Scrum Master: Dr. Anoop Kumar M**
- **Development Team Members:**

[1]. **BASIL ELDHO**

- Backend Development
- Idea pitching
- Documentation

[2]. **GINO PETER**

- Frontend Development
- Designing
- Documentation

[3]. **JESIL C J**

- Frontend Development
- Designing
- Documentation

VISION

UNEQUALLED SERVICES WITH UNFOLDING TECHNOLOGIES.

MISSION

Our mission is to:

1. FOSTER A CULTURE OF INNOVATION, CREATIVITY, AND ENTREPRENEURSHIP.
2. ENGAGE WITH INDUSTRY AND OTHER STAKEHOLDERS TO ADOPT LATEST TECHNOLOGICAL TRENDS AND DO RESEARCH.
3. IMBIBE PROFESSIONAL ETHICS, VALUES AND LIFE SKILLS.

PROGRAME EDUCATION OBJECTIVES

PEO 1:

Graduates will establish themselves as competent IT professionals, software developers, system analysts, or entrepreneurs by applying their technical knowledge, problem-solving skills, and innovative thinking in the field of computing and information technology.

PEO 2:

Graduates will uphold responsible research practices, promote innovative solutions and address societal challenges through technology-driven research, fostering inclusive development.

PEO 3:

Graduates will adhere and promote the ethical standards, cybersecurity principles, and sustainable computing practices while developing technology solutions for societal and business needs.

CHAPTER 1. PROJECT IDENTITY

This chapter presents the background, problem identification, proposed solution, comparative analysis, and functional use cases of the 3D House Floor Plan Designer Project. It establishes the need for an interactive web-based floor planning application and explains how the proposed system addresses limitations associated with traditional two-dimensional house planning methods. The chapter describes the problem addressed by the application, its relevance to society and the construction industry, the requirements identified for the system, the proposed solution, a comparison with existing approaches, and the principal user roles and use cases. The requirements tables, comparative analysis, and diagrams are intended for academic project documentation. Survey findings and implementation details should be updated according to actual project development and testing.

1.1 PROBLEM STORY

The 3D House Floor Plan Designer is a web-based application intended to simplify residential floor planning by enabling users to create, modify, and visualize house layouts in a three-dimensional environment. The application focuses on organizing rooms, arranging walls, placing doors and windows, positioning furniture, and examining the planned structure through interactive visualization.

1.1.1 PROBLEM STATEMENT

In modern residential construction and interior design, proper floor planning is essential for the effective utilization of available space. A well-designed floor plan helps determine room dimensions, establish suitable connections between different spaces, and organize furniture and architectural components according to the requirements of the occupants. However, designing and interpreting house layouts can be challenging for individuals who do not have professional architectural knowledge. Traditional two-dimensional floor planning applications represent rooms, walls, doors, windows, and furniture through flat drawings. Although these drawings provide important information about the layout and dimensions of a house, they may not clearly communicate the actual appearance, depth, height, and spatial relationships of architectural elements. Users without experience in interpreting architectural drawings may find it difficult to understand how the proposed design will appear in a real residential environment.

The major challenges associated with traditional floor planning methods include:

- Difficulty understanding the spatial arrangement of a house using only two-dimensional drawings.
- Limited visualization of the height, depth, and appearance of rooms and architectural components.
- Difficulty experimenting with different room layouts and furniture arrangements.
- Challenges in identifying space utilization issues before construction or renovation.
- Dependence on specialized architectural software that may be difficult for beginners to use.
- Difficulty communicating proposed house designs to homeowners, builders, interior designers, and other stakeholders.

Therefore, a user-friendly, interactive, and web-based floor planning application is required to help users create residential layouts, modify architectural elements, and visualize their designs in three dimensions.

Problem Statement: To design and develop a web-based 3D House Floor Plan Designer that enables users to create, edit, customize, and visualize residential floor plans through an interactive interface, supporting efficient space planning and improved understanding of house layouts before construction or renovation.

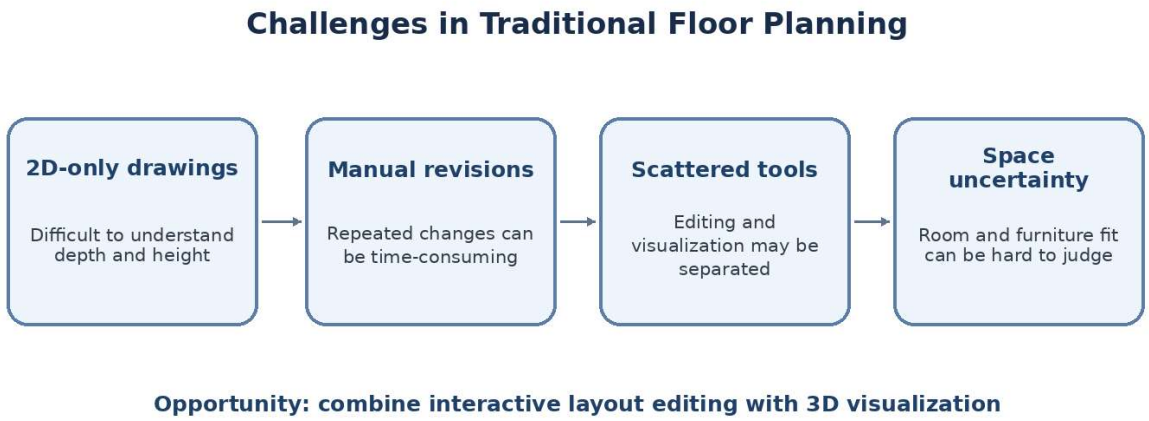


Figure 1.1: Problems Associated with Traditional Two-Dimensional Floor Planning

1.1.2 RELEVANCE TO SOCIETY / INDUSTRY

Relevance to homeowners: The platform enables homeowners to explore room arrangements, visualize proposed house layouts, and plan available residential space according to their requirements.

Relevance to architects and interior designers: The application can support preliminary layout development, furniture placement, and communication of design concepts to clients.

Relevance to the construction industry: Builders and construction professionals can use three-dimensional floor plan representations to understand proposed room arrangements and discuss potential design modifications.

Relevance to students and educational institutions: Students studying architecture, civil engineering, interior design, and related disciplines can use the platform to understand basic floor planning concepts and spatial relationships.

Relevance to society: A browser-based design platform can make basic residential planning more accessible to individuals who may not have access to advanced professional design software.

1.1.3 REQUIREMENTS SURVEYS

Requirements identification is an important stage in developing a user-oriented floor planning application. For the proposed 3D House Floor Plan Designer, requirements gathering should focus on understanding the needs of homeowners, students, interior designers, architects, and construction professionals.

Stakeholder	Information to be collected	Expected requirement
Homeowners	Difficulties understanding house layouts	Interactive 3D floor plan visualization
Architecture and design students	Challenges learning spatial planning	Simple room creation and editing tools

Interior designers	Requirements for arranging furniture	Furniture placement and layout customization
Architects	Requirements for preliminary residential layouts	Room dimensions and architectural element placement
Builders and contractors	Need to understand proposed layouts	Clear visualization of room arrangements

Table 1.1: Proposed Stakeholder Requirements Survey

The responses can be classified into functional and non-functional requirements.

ID	Requirement	Category	Priority
FR01	Create and display residential floor plans	Functional	High
FR02	Add and modify rooms and their dimensions	Functional	High
FR03	Place and edit walls, doors, and windows	Functional	High
FR04	Arrange furniture and interior elements	Functional	High
FR05	Generate a 3D representation of the floor plan	Functional	High
FR06	Interact with the 3D view using rotation and zoom	Functional	High
FR07	Modify existing floor plan layouts	Functional	High
NFR01	Provide a user-friendly interface	Non-functional	High
NFR02	Support responsive browser-based access	Non-functional	High
NFR03	Maintain acceptable rendering performance	Non-functional	High
NFR04	Maintain a modular and maintainable code structure	Non-functional	Medium
NFR05	Support commonly used modern web browsers	Non-functional	High

Table 1.2: Preliminary System Requirements

Documentation note: The survey questions and requirements above are proposed for the project. Actual survey findings, respondent counts, percentages, and final requirement priorities should be included only after the relevant information has been collected and evaluated.

1.2 PROJECT OVERVIEW – PROPOSED SOLUTION

1.2.1 SOLUTION CONCEPT

The proposed 3D House Floor Plan Designer is a web-based application designed to help users create, edit, and visualize residential floor plans through an interactive design environment. It combines floor plan editing functionality with three-dimensional visualization to make residential layout planning easier to understand. The system provides an interface through which users can organize rooms, define dimensions, place walls, add doors and windows, and arrange furniture within the available space. The resulting layout can be represented as a three-dimensional model, allowing users to examine the structure and understand the relationships between different architectural elements.

The application aims to reduce the difficulties associated with interpreting flat architectural drawings by providing a visual representation of the proposed residential design. The application can use modern web technologies for the user interface and a 3D graphics library such as **Three.js** for rendering architectural objects in a browser. **React.js** can be used to develop reusable interface components where appropriate. Additional backend services and database functionality may be incorporated if the project requires user accounts, saved designs, or persistent project management.

The system follows a modular design approach in which floor plan editing, architectural element management, and 3D visualization can be developed and maintained as separate functional components.

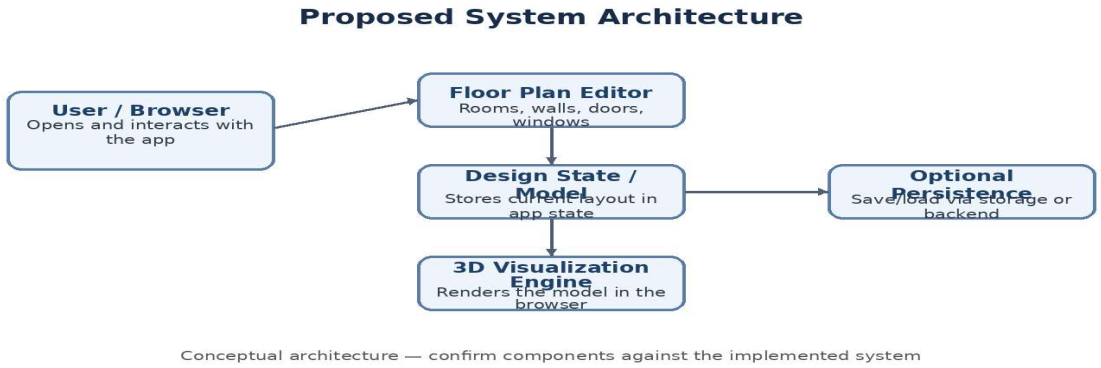


Figure 1.2: Proposed 3D House Floor Plan Designer System Architecture

1.2.1.1 WORKING PRINCIPLE

The proposed system follows a sequence of operations:

- The user opens the 3D House Floor Plan Designer through a compatible web browser.
- The application displays the floor plan design interface.
- The user creates a new floor plan and defines the required room layout.
- The user adds or modifies rooms, walls, doors, windows, and other supported architectural elements.
- The user arranges furniture and interior elements according to the available space.
- The application displays the corresponding three-dimensional representation of the floor plan.
- The user rotates, zooms, and examines the 3D model to understand the design from different perspectives.
- The user makes further modifications until the layout meets the intended requirements.

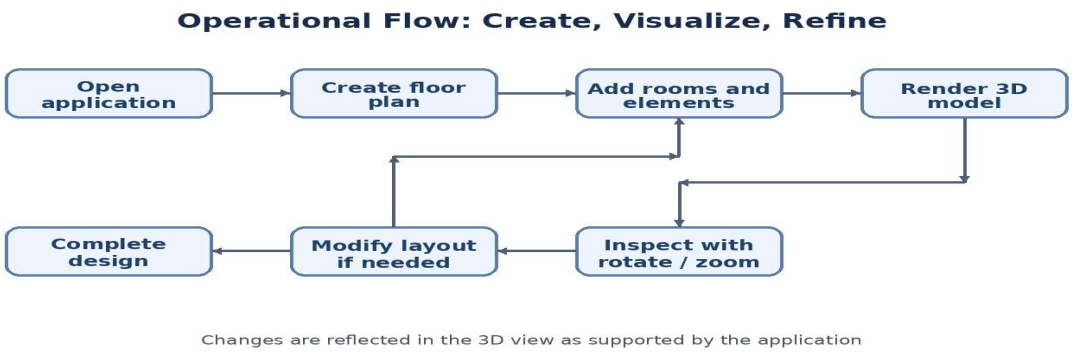


Figure 1.3: Operational Flow of the 3D House Floor Plan Designer

1.2.2 KEY FEATURES

Feature	Description	Expected Benefit
Interactive Design Interface	Provides controls for creating and modifying floor plans	Simplifies the design process
Room Creation and Editing	Supports arrangement and dimensioning of rooms	Helps organize residential space
Door and Window Placement	Allows openings to be positioned within the design	Supports planning of access and openings
Furniture Arrangement	Enables supported furniture objects to be placed within rooms	Helps evaluate space utilization
Real-Time 3D Visualization	Represents the floor plan as a 3D model	Improves understanding of the proposed structure
Interactive 3D Controls	Allows users to rotate and zoom the model	Supports inspection from different perspectives
Floor Plan Modification	Enables users to revise the existing layout	Supports experimentation with alternatives
Browser-Based Access	Provides access through a compatible browser	Reduces dependence on specialized desktop applications

Table 1.3: Feature List of Application

1.2.2.1 TECHNOLOGY REQUIREMENTS

Component	Proposed Technology	Purpose
User interface	HTML5, CSS3, JavaScript	Provides structure, styling, and interactive behavior
Frontend framework	React.js	Supports reusable interface components and state management
3D visualization	Three.js	Renders three-dimensional architectural elements in the browser
Development environment	Visual Studio Code or equivalent	Supports source-code development and debugging
Version control	Git and GitHub	Tracks code changes and support collaboration
Browser testing	Chrome, Firefox, Edge	Evaluates compatibility and interface behavior
Deployment	Suitable web hosting service	Makes the application accessible through a browser
Data storage	Local or backend storage	Supports persistence of floor plan information

Table 1.4: System Technology Requirements

1.2.2.2 NON-FUNCTIONAL FEATURES

Usability: The interface should provide clear controls and an understandable workflow for users with different levels of design experience.

Responsiveness: The interface should adapt to different screen sizes while maintaining access to the principal design and visualization controls.

Performance: The application should render the floor plan and supported 3D objects with acceptable responsiveness, including during interactive viewing.

Maintainability: The source code should be organized into logical components to simplify debugging, testing, and future modifications.

Compatibility: The application should operate correctly in the supported modern web browsers.

Accuracy of representation: Room dimensions and the relative placement of architectural elements should be represented consistently between the floor plan and its corresponding 3D view.

Accessibility: Text, controls, navigation, and other interface elements should be designed to remain understandable and usable by a broad range of users.

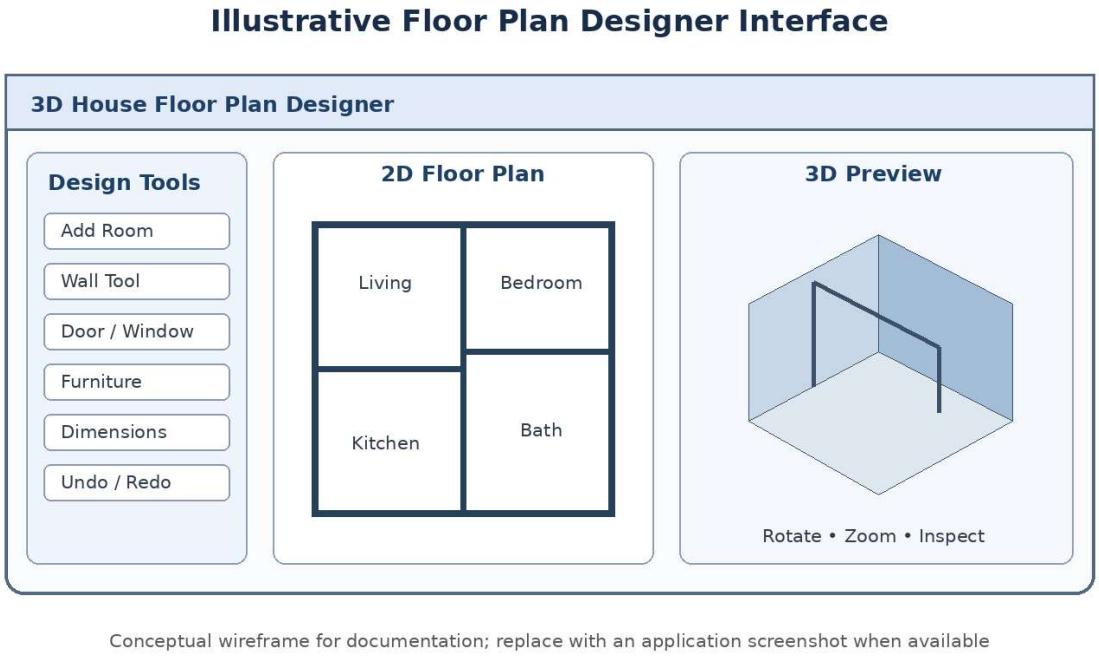


Figure 1.4: Illustrative Interface of the 3D House Floor Plan Designer

1.3 PROJECT ANALYSIS – EXISTING / SIMILAR VS PROPOSED

1.3.1 EXISTING / SIMILAR APPROACHES

Several methods and applications are available for residential floor planning and architectural visualization. These approaches differ in their visualization capabilities, ease of use, customization options, and technical requirements.

1.3.1.1 TRADITIONAL 2D FLOOR PLANNING

Traditional 2D floor planning represents rooms, walls, doors, and windows through flat drawings. It is useful for understanding room dimensions and the general arrangement of spaces. However, users may find it difficult to interpret height, depth, and the actual appearance of the proposed design from a two-dimensional representation alone.

1.3.1.2 MANUAL FLOOR PLAN DRAWING

Manual drawing methods use paper, pencils, rulers, and other drafting tools to prepare residential layouts. These methods can be useful for preliminary sketches, but modifying a design repeatedly may require redrawing portions of the plan. Manual drawings also do not inherently provide interactive three-dimensional visualization.

1.3.1.3 PROFESSIONAL ARCHITECTURAL DESIGN SOFTWARE

Professional architectural design applications provide extensive facilities for drafting, modelling, visualization, and technical documentation. However, their wide range of features may introduce a learning curve for beginners, and some applications may require installation, specialized hardware, or paid licenses.

1.3.1.4 SEPARATE 3D MODELLING TOOLS

General-purpose 3D modelling applications can represent buildings, rooms, and furniture as three-dimensional objects. Nevertheless, users may need to configure modelling tools and workflows specifically for floor planning, which can make the process less straightforward for individuals seeking a focused residential layout editor.

1.3.2 PROPOSED UNIQUENESS / IMPROVEMENT

The proposed 3D House Floor Plan Designer focuses on integrating floor plan editing and three-dimensional visualization within a single web-based environment. Its primary improvement is the combination of accessible layout creation, modification of architectural elements, and interactive 3D viewing in a focused application intended for residential planning.

Evaluation Criterion	Manual Drawing	Traditional 2D Planning	Professional Design Software	Proposed 3D Floor Plan Designer
Create room layouts	Yes	Yes	Yes	Intended
Modify room arrangements	Manual	Supported	Supported	Intended
Visualize layouts in 2D	Yes	Yes	Yes	Intended
Visualize layouts in 3D	Not inherently	Tool-dependent	Commonly supported	Core objective
Place furniture	Manual representation	Tool-dependent	Commonly supported	Intended
Interactive 3D viewing	No	Tool-dependent	Tool-dependent	Core objective
Browser-based access	N/A	Tool-dependent	Tool-dependent	Intended
Ease of use for beginners	Depends on drawing skills	Tool-dependent	May require training	Designed to be accessible
Specialized modelling capabilities	No	Limited/tool-dependent	Often extensive	Focused on supported floor planning features

Table 1.5: Comparative Analysis of Existing Approaches and the Proposed System

1.3.2.1 MAJOR IMPROVEMENTS OFFERED BY THE PROPOSED SYSTEM

Integrated floor plan creation: The application brings room layout editing and three-dimensional visualization into a single working environment.

Improved spatial understanding: Three-dimensional representation helps users understand the relationships between rooms, walls, openings, and furniture.

Interactive design modification: Users can experiment with different room arrangements and modify supported architectural elements.

Accessible interface: A browser-based approach can make the application available without requiring users to learn a complex professional modelling workflow.

Preliminary space planning: The system can help users evaluate room arrangements and furniture placement before proceeding to more detailed architectural design.

Scope for future extension: The modular application can potentially be extended with floor plan saving and loading, additional furniture libraries, measurement tools, multiple floors, and export functionality, depending on technical feasibility and project requirements.

1.4 USE CASES

Use-case analysis identifies the interactions between users and the proposed system. It helps establish the functional boundaries of the application and describes the services provided to its users.

In the proposed 3D House Floor Plan Designer, the primary user interacts with the floor plan editing interface to create or modify a residential layout and inspect the corresponding three-dimensional model. The availability of additional functions, such as saving projects or exporting designs, depends on the actual implementation.

1.4.1 IDENTIFIED USER ROLES

User Role	Description	Main Responsibilities / Interactions
Homeowner / General User	An individual planning a residential layout	Creates room arrangements and explores possible designs
Interior Designer	A professional planning interior arrangement	Evaluates room layouts and furniture placement
Architect / Design Professional	A professional preparing or reviewing preliminary layouts	Examines room dimensions and architectural arrangements
Builder / Contractor	A construction professional reviewing proposed layout	Uses the visual representation to understand the planned structure

Table 1.6: User Role Identification

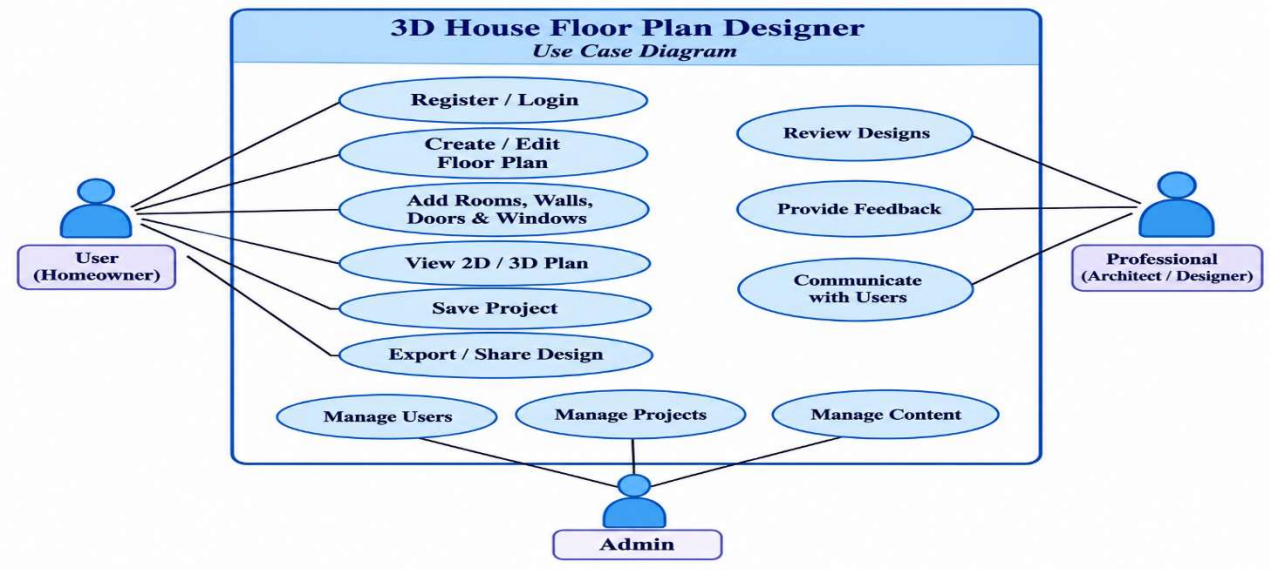


Figure 1.6: Use Case Diagram of the 3D House Floor Plan Designer

1.4.2 PRIMARY USE CASES

Use Case ID	Use Case Name	Primary Actor	Description
UC01	Open Design Interface	User	Accesses the floor plan design environment
UC02	Create Floor Plan	User	Begins creating a residential layout
UC03	Add Doors and Windows	User	Places supported openings in the layout
UC04	Arrange Furniture	User	Positions available furniture objects within rooms
UC05	View 3D Model	User	Examines the 3D representation of the design
UC06	Interact with 3D View	User	Rotates or zooms the model to inspect different perspectives
UC7	Modify Existing Floor Plan	User	Changes room arrangements and supported architectural elements
UC8	Save or Load Floor Plan	User	Stores or retrieves a design, if implemented
UC9	Export or Share Design	User	Exports or shares a floor plan, if implemented

Table 1.7: Primary Use Case Identification

1.4.3 USE CASE ACTIVITY FLOW

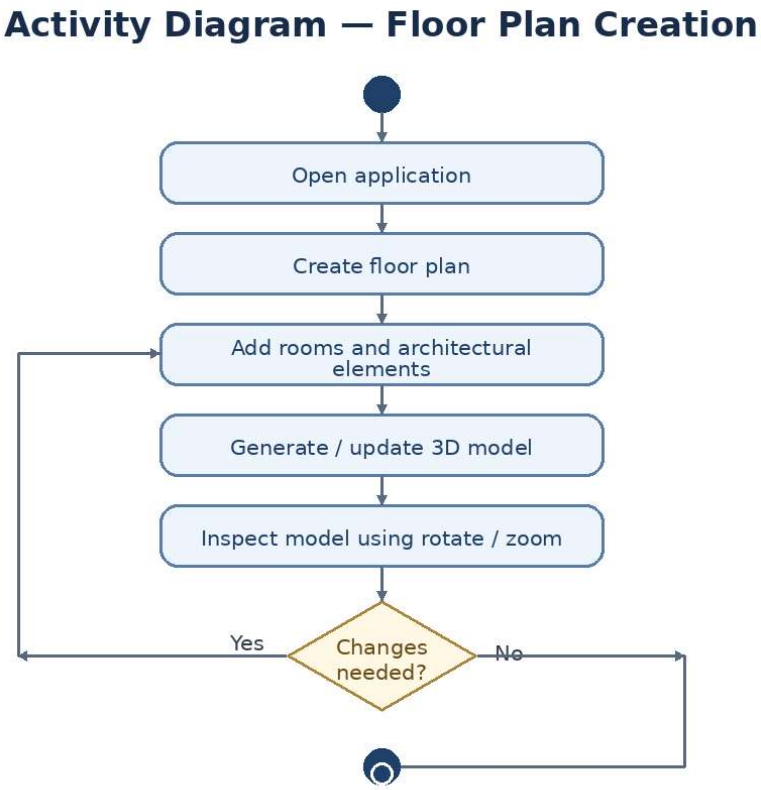


Figure 1.7: Activity Diagram for the 3D House Floor Plan Designer

CHAPTER 2. PRODUCT BACKLOG

2.1 OUTLINE

The Product Backlog acts as the central reference for all development activities in the 3D House Floor Plan Designer. It identifies the functionality required by users and organizes the work according to its importance, technical dependencies, and expected benefits.

The backlog includes the creation and modification of residential layouts, placement of walls and openings, furniture arrangement, interactive 3D visualization, and other supporting capabilities. It may evolve as the project progresses and additional requirements are identified.

2.1.1 WHAT BELONGS IN A PRODUCT BACKLOG

- Features: New capabilities such as creating rooms, editing dimensions, placing doors and windows, arranging furniture, and viewing the design in 3D.
- Bug Fixes: Defects related to room positioning, object placement, rendering, interaction controls, or interface responsiveness.
- Technical Improvements: Code refactoring, reusable components, 3D rendering optimization, and improvements to application architecture.
- Research and Learning: Investigations into suitable 3D libraries, object modelling, camera controls, coordinate systems, and techniques for representing architectural layouts.
- Usability Improvements: Enhancements to the interface, navigation, design controls, and overall user experience.

2.1.2 KEY CHARACTERISTICS

Detailed Appropriately: High-priority backlog items should be clearly defined and sufficiently small to implement and test. Future enhancements may remain less detailed until they are ready for development.

Emergent: The backlog can be updated based on user feedback, testing results, technical discoveries, and changes in project requirements.

Estimable: Development tasks should be sized according to their technical complexity and expected implementation effort.

Prioritized: Items should be ordered according to their importance, user value, implementation risks, and dependencies.

Transparent: Each backlog item should have a clear description, priority, owner or responsible team member, and implementation status.

2.1.3 CORE MANAGEMENT RULES

Ownership: The Product Owner or designated project lead maintains the backlog and determines the relative priority of requirements.

Estimation: The development team estimates the relative complexity of each Product Backlog Item (PBI), using story points or another agreed estimation method.

Refinement: Backlog items are reviewed regularly to clarify requirements, identify dependencies, and define acceptance criteria.

Sprint Selection: During Sprint Planning, the development team selects suitable high-priority items for implementation.

Validation: Completed features are tested against their acceptance criteria before being considered finished.

2.2 INTEGRATED INITIAL RECORD

The initial product backlog for the 3D House Floor Plan Designer identifies the major features required to create and visualize residential floor plans.

ID	User Story / Feature	Priority	Status
US01	Open the floor plan design interface	Must	Proposed
US02	Create a new residential floor plan	Must	Proposed
US03	Add and edit walls	Must	Proposed
US04	Place doors and windows	Must	Proposed
US05	Arrange furniture and interior elements	Should	Proposed
US06	Generate a 3D representation of the floor plan	Must	Proposed
US07	Rotate and zoom the 3D model	Must	Proposed
US8	Modify an existing floor plan	Must	Proposed
US9	Navigate the design interface smoothly	Must	Proposed
US10	Use a responsive browser-based interface	Must	Proposed
US11	Save and load floor plans	Could	Proposed
US12	Optimize 3D rendering and loading performance	Should	Proposed

Table 2.1: List of User Stories and Features Prioritization

- Must Have: US01, US02, US03, US04, US05, US06, US08, US09, US10.
- Should Have: US05, US11, US12.
- Could Have: US11.

2.3 USE CASE UC-01 — OPEN DESIGN INTERFACE

Backlog Mapping: US01

Priority: Must

Primary Actor: Homeowner / General User

2.3.1 USER STORY

As a user, I want to open the floor plan design interface so that I can access the tools required to create and visualize a residential layout.

2.3.2 OBJECTIVE

To provide a clear and accessible starting point for creating, editing, and viewing a house floor plan.

2.3.3 TRIGGER

The user opens the 3D House Floor Plan Designer through a supported web browser.

2.3.4 PRECONDITIONS

1. The application is available through a compatible browser.
2. The required interface resources can be loaded.
3. The browser supports the application's essential rendering and interaction features.

2.3.5 MAIN SUCCESS SCENARIO

Step	Actor / System Action
1	The user opens the application URL.
2	The system loads the main design interface.
3	The system displays the available floor plan tools and controls.
4	The system initializes the floor plan workspace and 3D viewing area.
5	The user reviews the available controls.
6	The user begins creating or modifying a floor plan.

2.3.6 ALTERNATE / EXCEPTION FLOW

If a required resource fails to load, the application should display an appropriate error message where feasible. The interface should avoid displaying misleading or unusable controls.

2.3.7 POSTCONDITION

The user can access the floor plan design environment.

2.3.8 ACCEPTANCE CRITERIA

- The design interface opens successfully.
- The main controls are clearly identifiable.
- The workspace is displayed without major layout errors.
- The interface supports the intended browser environment.

2.4 USE CASE UC-02 — CREATE A NEW FLOOR PLAN

Backlog Mapping: US02

Priority: Must

Primary Actor: Homeowner / General User

2.4.1 USER STORY

As a user, I want to create a new floor plan so that I can begin designing a residential layout according to my requirements.

2.4.2 OBJECTIVE

To allow users to initiate a new design and establish the initial layout for the proposed house.

2.4.3 TRIGGER

The user selects the option to create or start a new floor plan.

2.4.4 PRECONDITIONS

- The design interface is available.
- The user can access the floor plan creation controls.

2.4.5 MAIN FLOW

1. The user selects the new floor plan option.

2. The system initializes an empty or default design workspace.
3. The user begins arranging the initial room layout.
4. The system displays the created layout in the workspace.
5. The user continues adding rooms and architectural elements.

2.4.6 ALTERNATE FLOW

If the application uses a predefined starting layout, the user can modify that layout instead of starting with an empty workspace.

2.4.7 POSTCONDITION

A new floor plan is available for editing.

2.4.8 ACCEPTANCE CRITERIA

- A new design workspace can be initialized.
- The workspace is ready for room and element placement.
- The initial design is displayed clearly.
- The creation process does not interfere with the primary interface.

2.5 USE CASE UC-03 — ADD AND EDIT WALLS

Backlog Mapping: US03

Priority: Must

Primary Actor: Homeowner / General User

2.5.1 USER STORY

As a user, I want to add and edit walls so that I can define room boundaries and the structural arrangement of the planned house.

2.5.2 OBJECTIVE

To represent the boundaries and divisions between rooms in the residential floor plan.

2.5.3 MAIN FLOW

1. The user selects the wall creation or editing tool.
2. The user specifies the wall's position and supported dimensions.
3. The system creates or updates the wall.
4. The system reflects the change in the floor plan.
5. The 3D representation is updated where supported.
6. The user examines the resulting layout.

2.5.4 ALTERNATE FLOW

If the wall placement conflicts with application constraints, the system should prevent invalid changes or provide suitable feedback.

2.5.5 ACCEPTANCE CRITERIA

- Users can create supported wall elements.
- Wall positions and dimensions are represented consistently.
- Wall modifications update the relevant views.
- Wall geometry does not introduce avoidable rendering errors.

2.6 USE CASE UC-04 — PLACE DOORS AND WINDOWS

Backlog Mapping: US04

Priority: Must

Primary Actor: Homeowner / General User

2.6.1 USER STORY

As a user, I want to place doors and windows so that I can represent openings and access points in the proposed house layout.

2.6.2 OBJECTIVE

To allow the placement of supported architectural openings within appropriate wall locations.

2.6.3 MAIN FLOW

1. The user selects the door or window tool.
2. The user chooses the required element.
3. The user positions the element in the design.
4. The system updates the floor plan representation.
5. The system reflects the element in the 3D view where supported.
6. The user reviews the result.

2.6.4 ACCEPTANCE CRITERIA

- Supported doors and windows can be selected.
- Their placement is visually represented.
- The element is associated with the intended wall or location where required.
- The floor plan and 3D representation remain consistent.

2.7 USE CASE UC-05 — ARRANGE FURNITURE AND INTERIOR ELEMENTS

Backlog Mapping: US05

Priority: Should

Primary Actor: Homeowner / Interior Designer

2.7.1 USER STORY

As a user, I want to arrange furniture and interior elements so that I can evaluate the utilization of available space.

2.7.2 OBJECTIVE

To support preliminary furniture placement and help users understand how interior objects fit within the planned rooms.

2.7.3 MAIN FLOW

1. The user opens the available furniture or object selection tool.
2. The system displays supported furniture items.
3. The user selects an object.
4. The user places it within a room.
5. The system displays the object in the design workspace.
6. The user adjusts the object's position or orientation where supported.
7. The user examines the resulting arrangement.

2.7.4 ACCEPTANCE CRITERIA

- Supported furniture objects can be selected and placed.
- Furniture placement is clearly visible.
- Objects can be repositioned where supported.
- The arrangement can be inspected in the relevant design view.
- The furniture representation does not unnecessarily obstruct the main interface.

2.8 USE CASE UC-06 — GENERATE A 3D REPRESENTATION

Backlog Mapping: US06

Priority: Must

Primary Actor: Homeowner / General User

2.8.1 USER STORY

As a user, I want to view the floor plan in three dimensions so that I can better understand the appearance and spatial relationships of the proposed house.

2.8.2 OBJECTIVE

To convert the supported floor plan elements into a three-dimensional visual representation.

2.8.3 MAIN FLOW

1. The user creates or modifies a floor plan.
2. The system reads the current room and architectural element information.
3. The system generates the corresponding 3D geometry.
4. The system displays the model in the visualization area.
5. The user examines the rooms, walls, openings, and supported furniture.
6. The user returns to editing if additional changes are required.

2.8.4 ALTERNATE FLOW

If an element cannot be represented in 3D, the system should handle the unsupported element without causing the entire visualization to fail.

2.8.5 ACCEPTANCE CRITERIA

- A 3D representation can be generated from the supported floor plan data.
- The model includes the architectural elements implemented by the project.
- The 3D view reflects relevant changes to the design.
- The model can be displayed without major rendering errors.
- The representation remains understandable to users.

2.8.6 BUSINESS VALUE

Three-dimensional visualization helps users understand a proposed residential arrangement more intuitively than a flat drawing alone. It also supports discussion of preliminary design alternatives before construction or renovation.

2.9 USE CASE UC-07 — INTERACT WITH THE 3D VIEW

Backlog Mapping: US07

Priority: Must

Primary Actor: Homeowner / General User

2.9.1 USER STORY

As a user, I want to rotate and zoom the 3D model so that I can inspect the floor plan from different viewpoints.

2.9.2 OBJECTIVE

To provide interactive controls for examining the spatial layout and architectural components of the design.

2.9.3 MAIN FLOW

1. The user opens the 3D visualization.
2. The system displays the current model.
3. The user rotates the camera or model using the available controls.
4. The user zooms in or out to examine details.
5. The system updates the displayed viewpoint.
6. The user continues inspecting the model.

2.9.4 ACCEPTANCE CRITERIA

- Rotation and zoom controls operate as implemented.
- The user can inspect the model from multiple viewpoints.
- Camera interaction does not unintentionally modify the floor plan.
- The 3D view remains responsive during normal interaction.
- The controls are understandable and accessible.

2.10 USE CASE UC-08 — MODIFY AN EXISTING FLOOR PLAN

Backlog Mapping: US08

Priority: Must

Primary Actor: Homeowner / General User

2.10.1 USER STORY

As a user, I want to modify an existing floor plan so that I can experiment with alternative room arrangements and improve the proposed design.

2.10.2 OBJECTIVE

To allow iterative design changes without requiring the user to recreate the entire floor plan.

2.10.3 MAIN FLOW

1. The user opens an existing design in the current workspace, where supported.
2. The user selects a room or architectural element.

3. The user changes its position, dimensions, or supported properties.
4. The system updates the floor plan.
5. The system updates the 3D representation where applicable.
6. The user reviews the revised design.

2.10.4 ACCEPTANCE CRITERIA

- Supported elements can be modified after creation.
- Changes are reflected in the relevant views.
- The interface handles invalid modifications appropriately.
- The user can continue editing without restarting the entire design process.

2.11 USE CASE UC-09 — NAVIGATE THE DESIGN INTERFACE

Backlog Mapping: US09

Priority: Must

Primary Actor: Homeowner / General User

2.11.1 USER STORY

As a user, I want to navigate the design interface smoothly so that I can locate the required tools and complete floor planning tasks efficiently.

2.11.2 OBJECTIVE

To provide a consistent interface for accessing design tools, editing controls, and 3D visualization features.

2.11.3 MAIN FLOW

1. The user opens the application.
2. The system displays the main interface and available controls.
3. The user selects a design tool or visualization option.
4. The system displays the corresponding functionality.
5. The user switches between editing and 3D viewing as supported.
6. The user continues working on the design.

2.11.4 ACCEPTANCE CRITERIA

- Main tools are clearly identifiable.
- Navigation and control labels are understandable.
- Switching between supported functions works correctly.
- No major interface element prevents access to essential features.
- The interface provides a consistent experience across supported browsers.

2.12 USE CASE UC-10 — USE A RESPONSIVE BROWSER-BASED INTERFACE

Backlog Mapping: US10

Priority: Must

Primary Actor: Homeowner / General User

2.12.1 USER STORY

As a user, I want to access the application through a compatible browser and use an interface suited to my device so that I can explore floor plans conveniently.

2.12.2 OBJECTIVE

To make the principal design controls and visualization features usable on supported desktop and mobile screen sizes.

2.12.3 MAIN FLOW

1. The user opens the application on a supported device.
2. The system adapts the interface to the available viewport.
3. The system adjusts the placement and size of controls.
4. The user accesses the available design tools.
5. The user creates or inspects the floor plan.

2.12.4 ACCEPTANCE CRITERIA

- Main interface elements fit the supported viewport.
- Text and controls remain readable.
- Essential controls remain accessible.
- The 3D viewing area remains usable on supported devices.

2.14.5 QUALITY ATTRIBUTE

Responsiveness is a cross-cutting requirement that influences the design interface, navigation, room editing controls, and 3D visualization area.

2.13 USE CASE UC-11 — SAVE AND LOAD FLOOR PLANS

Backlog Mapping: US11

Priority: Could

Primary Actor: Homeowner / General User

2.13.1 USER STORY

As a user, I want to save and load floor plans so that I can continue working on a design at a later time.

2.13.2 OBJECTIVE

To provide persistent access to user-created layouts if storage functionality is included in the project scope.

2.13.3 MAIN FLOW

1. The user creates or modifies a floor plan.
2. The user selects the save option.
3. The system stores the supported design information using the implemented storage mechanism.
4. The user later selects the load option.
5. The system retrieves the saved design.
6. The system reconstructs the floor plan and corresponding visualization where supported.

2.13.4 ALTERNATE FLOW

If saving or loading is not implemented, this use case remains a proposed enhancement rather than a completed application capability.

2.13.5 ACCEPTANCE CRITERIA

- Saved design data can be retrieved if persistence is implemented.
- Room and architectural element information is preserved.
- Invalid or unavailable design files are handled appropriately.
- Loading a design does not produce major rendering errors.

2.14 USE CASE UC-12 — OPTIMIZE 3D RENDERING AND LOADING PERFORMANCE

Backlog Mapping: US12

Priority: Should

Primary Actor: Developer / General User

2.14.1 USER STORY

As a user, I want the application to render floor plans efficiently so that I can interact with the design without unnecessary delays.

2.14.2 OBJECTIVE

To maintain acceptable responsiveness during interface loading, floor plan editing, and 3D visualization.

2.14.3 MAIN FLOW

1. The developer identifies performance bottlenecks.

2. The developer examines geometry complexity, image assets, and rendering operations.
3. The developer applies appropriate optimizations.
4. The application is tested using representative floor plans.
5. The developer checks rendering responsiveness and visual correctness.
6. The application is refined where necessary.

2.14.4 ACCEPTANCE CRITERIA

- Unnecessary resources and operations are minimized where feasible.
- Common floor plan operations remain responsive under the tested conditions.
- 3D rendering does not introduce avoidable delays.
- Optimization does not compromise essential functionality.
- The application remains stable during normal design interaction.

2.15 SUGGESTED ACTOR MODEL

The actors for the UML Use Case Diagram can be organized according to their interaction with the 3D House Floor Plan Designer.

Primary Actor

- Homeowner / General User: Creates and modifies residential layouts and examines the resulting 3D representation.

Supporting Actors

- Interior Designer: Uses the application for preliminary room and furniture arrangement.
- Architect / Design Professional: Reviews room dimensions and architectural layout.
- Developer: Implements, tests, and maintains the application.

High-level actor relationships

Homeowner / General User → 3D House Floor Plan Designer

- Open Design Interface
- Create Floor Plan
- Add and Modify Rooms
- Define Room Dimensions
- Add and Edit Walls
- Place Doors and Windows
- Arrange Furniture
- Generate 3D Representation
- Interact with 3D View
- Modify Existing Floor Plan
- Navigate the Design Interface

Interior Designer / Architect → 3D House Floor Plan Designer

- Create or Review Room Arrangements
- Examine Architectural Elements
- Arrange Furniture, where supported
- Inspect the 3D Representation

Developer → 3D House Floor Plan Designer

- Maintain Reusable Components
- Optimize Rendering Performance
- Test Application Behaviour
- Maintain Source Code and Version History, where configured

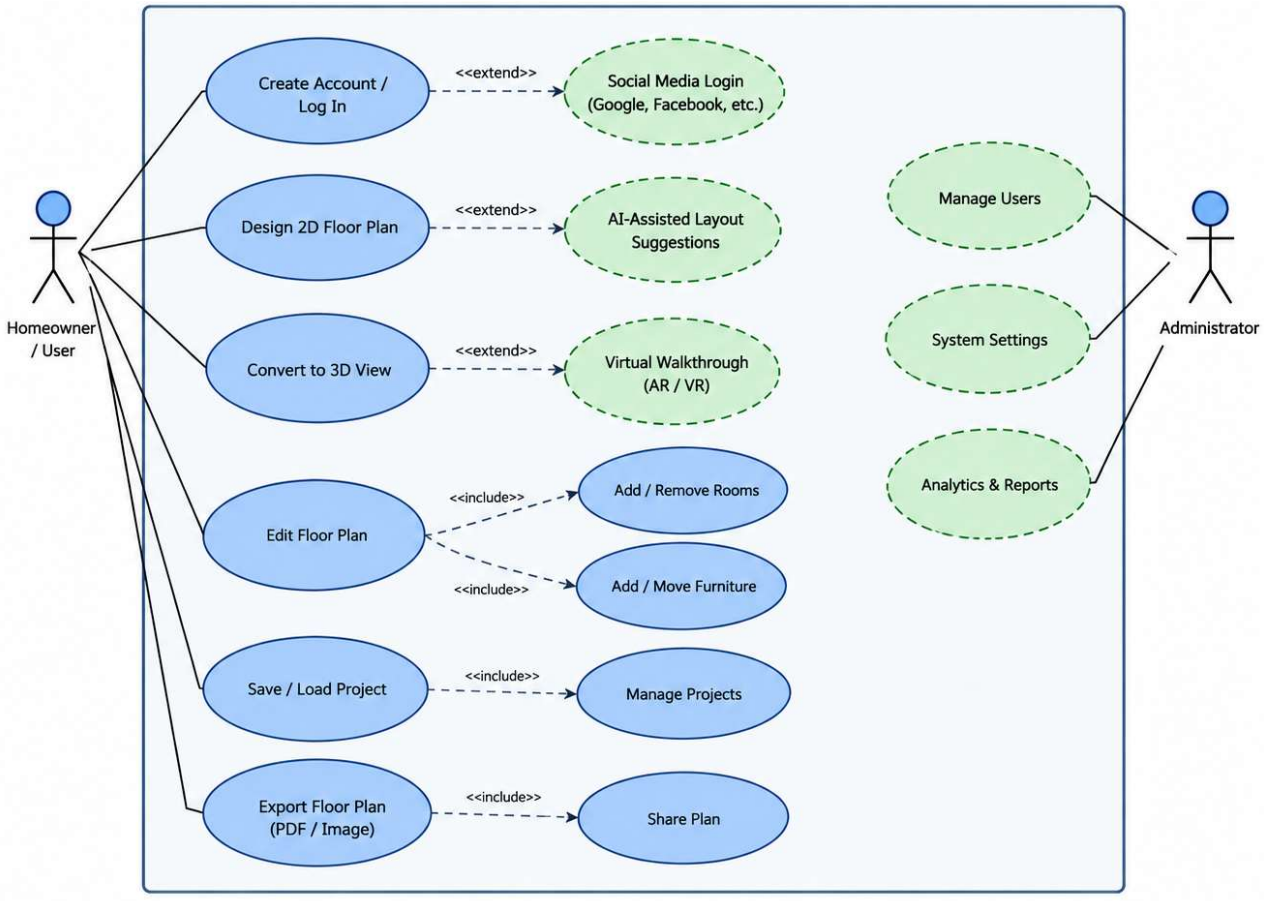


Figure 2.1: High-Level Use Case Diagram of the 3D House Floor Plan Designer.

CHAPTER 3. STORY POINT ESTIMATION

3.1 INTRODUCTION

Story point estimation is an important activity in Agile software development that helps the development team estimate the relative effort, complexity, and uncertainty associated with implementing individual user stories. In the 3D House Floor Plan Designer project, story points are used to estimate the work required to implement the user stories identified in the product backlog.

Unlike conventional time-based estimation, story points represent the relative size of a task rather than a guaranteed number of working hours. The estimation considers factors such as functional complexity, implementation effort, technical dependencies, testing requirements, and uncertainty. These estimates help the development team prioritize features, allocate responsibilities, plan sprint workloads, and monitor progress throughout the project lifecycle.

For the 3D House Floor Plan Designer, a modified Fibonacci-based estimation scale consisting of 1, 2, 3, 5, and 8 story points can be adopted. Smaller values represent straightforward tasks, whereas larger values indicate greater implementation complexity or uncertainty. The estimates should reflect the actual functionality and acceptance criteria of the application.

3.2 OBJECTIVES OF STORY POINT ESTIMATION

The primary objectives of story point estimation in the 3D House Floor Plan Designer are:

1. To estimate the relative complexity of each user story before implementation.
2. To support prioritization and sprint planning based on user value and implementation effort.
3. To distribute responsibilities among team members according to the identified workload.
4. To identify technically demanding features that require additional design, implementation, or testing.
5. To compare estimated effort with actual progress during sprint reviews and retrospectives.
6. To establish a measurable basis for monitoring team velocity and planning subsequent iterations.
7. To identify dependencies between floor plan editing, architectural elements, and 3D visualization.
8. To distinguish essential application functionality from proposed enhancements.

3.3 ESTIMATION APPROACH AND SCALE

The development team uses relative estimation to assign story points to each user story. The estimation scale helps distinguish simple interface-related tasks from more complex features involving room geometry, architectural elements, interactive 3D visualization, or performance optimization.

Story Points	Relative Complexity	Typical Interpretation
1	Very small	A minor change with minimal implementation effort
2	Small	A straightforward feature with limited complexity
3	Moderate	A feature requiring several implementation steps
5	High	A feature involving multiple components, interactions, or significant testing
8	Very high	A comparatively complex feature with substantial uncertainty or technical dependencies

Table 3.1: Story Point Estimation Scale

3.4 USER STORY ESTIMATION REGISTER

The product backlog for the 3D House Floor Plan Designer identifies twelve principal user stories. These stories cover the main design workflow and supporting technical improvements. The following register provides a suggested estimation structure for planning purposes.

ID	User Story / Feature	Priority	Suggested SP	Status
US01	Open the floor plan design interface	Must	2	Proposed
US02	Create a new residential floor plan	Must	3	Proposed
US03	Add and edit walls	Must	5	Proposed
US04	Place doors and windows	Must	3	Proposed
US05	Arrange furniture and interior elements	Should	5	Proposed
US06	Generate a 3D representation of the floor plan	Must	8	Proposed
US07	Rotate and zoom the 3D model	Must	3	Proposed
US08	Modify an existing floor plan	Must	5	Proposed
US09	Navigate the design interface smoothly	Must	2	Proposed
US10	Use a responsive browser-based interface	Must	5	Proposed
US11	Save and load floor plans	Could	5	Proposed enhancement
US12	Optimize 3D rendering and loading performance	Should	5	Proposed

Table 3.2: Story Point Estimation

The estimates above are suggested values for planning. They should be reviewed against the actual implementation before being treated as accepted estimates. The status "Proposed" indicates that the backlog entry itself does not establish that the corresponding functionality has been implemented and tested.

3.5 RATIONALE FOR STORY POINT ASSIGNMENT

The suggested estimates reflect the relative complexity of the functionality, the expected implementation activities, and the testing required to verify each feature.

User Story	SP	Justification
US01 – Open design interface	2	Primarily involves initializing the interface and presenting the available design controls.
US02 – Create a new floor plan	3	Requires initializing a workspace and making it ready for layout creation.
US03 – Add and edit walls	5	Involves wall positioning, supported dimensions, geometry updates, and validation.
US04 – Place doors and windows	3	Requires selecting architectural openings and representing their positions in the layout.
US05 – Arrange furniture and interior elements	5	Involves object selection, placement, positioning, and checking spatial arrangements.
US06 – Generate a 3D representation	8	Requires converting supported layout elements into 3D geometry and rendering the result.
US07 – Rotate and zoom the 3D model	3	Requires interactive camera controls and verification of the viewing experience.
US08 – Modify an existing floor plan	5	Requires updating existing elements and ensuring relevant views remain consistent.
US09 – Navigate the design interface	2	Focuses on accessible controls and movement between supported design functions.
US10 – Responsive browser interface	5	Requires layout adaptation and usability testing at different viewport sizes.
US11 – Save and load floor plans	5	May require data serialization, storage, retrieval, and reconstruction of the layout.
US12 – Optimize rendering performance	5	Involves identifying bottlenecks, optimizing resources, and testing rendering responsiveness.

3.6 STORY POINT DISTRIBUTION ANALYSIS

Story point distribution helps the team understand the relative size of the work represented by the product backlog. Based on the suggested estimates in Table 3.2, the distribution is as follows.

Table 3.4: Distribution of User Stories by Story Point Estimate

Story Points	Number of User Stories	Total Story Points
2	2	4
3	3	9
5	6	30
8	1	8
Total	12	51

The largest group consists of six user stories estimated at five points. These include wall editing, furniture arrangement, modifying an existing floor plan, responsive interface support, saving and loading, and performance optimization. Such features generally require several implementation or verification activities.

The 3D representation feature receives the highest suggested estimate because it may involve geometry generation, rendering, and integration with the floor plan data. Its final estimate should depend on the complexity of the application's actual 3D implementation.

3.6.1 PRIORITY DISTRIBUTION

Priority	Number of Stories	Total Story Points	Percentage of Total Points
Must	8	31	60.78%
Should	3	13	25.49%
Could	1	5	9.80%
Total	12	51	100%

Table 3.5: User Stories by Priority

3.7 TEAM-WISE WORKLOAD DISTRIBUTION

Team-wise workload distribution helps identify responsibilities and supports balanced sprint planning. The supplied project backlog does not specify named team members or their story ownership. Therefore, individual workload figures cannot be established from the available information without making assumptions The development team can use the following template to record assignments after responsibilities have been agreed.

Team Member	Assigned Stories	Story Points	Estimated Hours
Basil Eldho	4	17	22
Gino Peter	4	17	22
Jesil C J	4	17	22
Total	12	51	66

Table 3.6: Team-Wise Story Point and Hour Distribution

Work may be divided among interface development, floor plan editing, architectural elements, 3D visualization, testing, and performance improvement. Integration and validation may require collaboration across team members.

3.8 ESTIMATED EFFORT AND STORY POINT RELATIONSHIP

The 3D House Floor Plan Designer Project contains a total of 51 story points and 66 estimated hours. These estimates represent the relative complexity of the user stories and the preliminary effort expected for implementing the project's features, such as floor plan creation, wall editing, door and window placement, furniture arrangement, and 3D visualization.

The indicative relationship between estimated hours and story points is calculated as follows:

Indicative hours per story point

$$\frac{\text{Total Estimated Hours}}{\text{Total Story Points}} = \frac{66}{51} \approx 1.29$$

Therefore, the indicative ratio is approximately 1.29 estimated hours per story point.

3.9 USE OF ESTIMATES IN SPRINT PLANNING

The story point estimates provide a starting point for planning development iterations for the 3D House Floor Plan Designer. The team should consider priority, technical dependencies, story readiness, testing requirements, and available capacity before selecting stories for a sprint.

The recommended sprint planning process is:

1. Review the product backlog and confirm the acceptance criteria for each user story.
2. Identify dependencies, such as creating a floor plan before generating its 3D representation.
3. Select high-priority stories that fit the sprint objective and available team capacity.
4. Assign implementation and verification responsibilities.
5. Monitor progress using a sprint board and regular team discussions.
6. Review completed functionality against acceptance criteria during the sprint review.
7. Record estimation differences, impediments, and improvement actions during the retrospective.
8. Update the backlog when testing or user feedback identifies new requirements.

A user story should be marked Accepted only when its defined acceptance criteria have been satisfied. Proposed enhancements, such as saving and loading floor plans, should remain identified as future work until their implementation and testing have been verified.

CHAPTER 4. RELEASE & SPRINT PLAN

4.1 INTRODUCTION

This chapter presents the release strategy, prioritized product backlog, sprint-wise implementation schedule, design considerations, architectural overview, logical data model, UI/UX wireframe plan, and technology stack of the 3D House Floor Plan Designer Project. The project aims to develop a web-based application that enables users to create, modify, and visualize residential floor plans through an interactive interface. The application is intended to simplify house planning by allowing users to arrange rooms, edit walls, place doors and windows, arrange furniture, and view the proposed layout as a three-dimensional model.

The development plan is organized into six proposed sprints over twelve weeks. The initial sprints focus on requirements analysis, project setup, and floor plan creation. The subsequent sprints cover architectural element placement, 3D visualization, editing features, responsive interface design, performance improvements, testing, final release preparation and documentation.

4.2 PRODUCT BACKLOG PRIORITIZATION AND ESTIMATION

The product backlog contains the functional and quality requirements identified for the 3D House Floor Plan Designer Project. The MoSCoW prioritization method is used to categorize requirements into Must, Should, and Could groups.

ID	User Story / Feature	Priority	SP	Hours
US01	Open the design interface	Must	2	3
US02	Create a new floor plan	Must	3	4
US03	Add and edit walls	Must	5	7
US04	Place doors and windows	Must	3	4
US05	Arrange furniture and interior elements	Should	5	6
US06	Generate a 3D representation	Must	8	10
US07	Rotate and zoom the 3D model	Must	3	4
US08	Modify an existing floor plan	Must	5	7
US09	Navigate the design interface	Must	2	3
US10	Support a responsive browser interface	Must	5	7
US11	Save and load floor plans	Could	5	6
US12	Optimize 3D rendering and loading performance	Should	5	5
Total	12 user stories		51	66

Table 4.1: Prioritized Product Backlog

.

4.3 PLANNED DETAILED SCHEDULE

The release plan distributes the twelve user stories across six proposed sprints. The sequence prioritizes the basic interface and floor plan creation before adding architectural elements, 3D visualization, editing functionality, and optional enhancements.

Sprint	Planned Focus	User Stories	SP	Hours
Sprint 1	Interface, floor plan creation, navigation and responsive foundation	US01, US02, US09, US10	12	17
Sprint 2	Wall editing and placement of doors and windows	US03, US04	8	11
Sprint 3	Furniture arrangement and editing existing floor plans	US05, US08	10	13
Sprint 4	3D visualization and model navigation	US06, US07	11	14
Sprint 5	Save and load floor plans	US11	5	6
Sprint 6	3D rendering and loading optimization	US12	5	5
Total	12 user stories	US01–US12	51	66

Table 4.2: Sprint-Wise Implementation Schedule

4.3.1 SPRINT ACTIVITIES AND DELIVERABLES

Sprint	Main Activities	Expected Deliverables
Sprint 1	Requirements, project setup, interface and navigation.	Basic floor plan editor and responsive interface.
Sprint 2	Create walls, doors, and windows.	Floor plan with architectural elements.
Sprint 3	Arrange furniture and edit floor plans.	Editable floor plan with interior elements.
Sprint 4	Generate 3D models; implement rotation and zoom.	Interactive 3D floor plan viewer.
Sprint 5	Implement saving and loading, if included.	Save/load functionality, if implemented.
Sprint 6	Optimize rendering, test, and finalize documentation.	Tested release candidate and final report.

Table 4.3: Sprint Schedule

4.3.2 RELEASE MILESTONES

1. Milestone 1 – Project Foundation Ready:

Completion of the initial design interface, floor plan creation foundation, navigation, and responsive layout work.

2. Milestone 2 – Architectural Elements Complete:

Implementation of wall editing and placement of doors and windows.

3. Milestone 3 – Floor Plan Editing Complete:

Support for furniture arrangement and modification of existing floor plans.

4. Milestone 4 – 3D Visualization Complete:

Generation of the 3D model and support for rotation and zoom operations.

5. Milestone 5 – Optional Persistence Feature:

Completion of save/load functionality if it is included in the approved project scope.

6. Milestone 6 – Final Release:

Completion of performance checks, regression testing, final integration, and project documentation.

4.4 DESIGN ASPECTS FOR PRIORITIZED REQUIREMENTS

The design of the 3D House Floor Plan Designer is guided by the prioritized user stories and their acceptance criteria. The interface should allow users to understand the layout, place architectural elements accurately, and inspect the resulting model with minimal difficulty.

Requirement Group	User Stories	Design Consideration
Design interface and navigation	US01, US09	Clear toolbar, understandable controls, consistent navigation, and visible editing options.
Floor plan creation	US02	Structured drawing area, suitable scale, and clear floor plan boundaries.
Walls and architectural elements	US03, US04	Accurate positioning, consistent dimensions, and appropriate placement of doors and windows.
Furniture and interior arrangement	US05	Easy placement and repositioning of furniture and interior objects.
3D visualization	US06	Clear three-dimensional representation of walls, rooms, and architectural elements.
3D model navigation	US07	Intuitive rotation and zoom controls for examining the model from different viewpoints.
Floor plan modification	US08	Editing controls that allow users to revise the layout without recreating the entire design.

Responsive interface	US10	Flexible layouts and usable controls across supported desktop, tablet, and mobile screen sizes.
Save/load functionality	US11	Clear save/load operations, appropriate storage handling, and feedback about successful or unsuccessful operations.
Rendering performance	US12	Efficient model rendering, optimized assets, and reasonable loading and interaction times.

Table 4.4: Design Mapping for Prioritized Requirements

The interface should maintain consistent spacing, readable typography, clear labels, and visible feedback for user actions. The floor plan editor and 3D viewer should be organized so that users can switch between creating a layout and examining its three-dimensional appearance.

4.5 ARCHITECTURAL OVERVIEW

The 3D House Floor Plan Designer follows a layered web application architecture. The application uses React, TypeScript, and Vite to develop the browser-based interface, while Three.js supports three-dimensional visualization of house floor plans. Node.js and Express.js provide backend services, and MongoDB with Mongoose supports data storage and management.

The frontend layer is responsible for displaying the dashboard, floor plan editor, design controls, and 3D viewer. The backend layer handles application logic, authentication, project management, and supported sharing operations. The database layer stores user information and project data, enabling floor plans to be saved and retrieved. Additional components support blueprint import, plan export, messaging, and engineer review where implemented. These layers work together to provide an integrated environment for creating, editing, visualizing, and managing house floor plans.

4.6 LOGICAL DATA MODEL / ER CONCEPT

The logical data model describes how floor plan information could be organized if persistent storage is implemented. It identifies the main entities and their relationships for representing a residential layout.

Entity	Representative Attributes	Purpose
FloorPlan	plan_id, name, dimensions, created_date	Represents a residential floor plan.
Room	room_id, plan_id, room_type, dimensions	Stores room types and their dimensions.
Wall	wall_id, plan_id, position, length, height	Represents wall segments within the layout.
Door/Window	element_id, wall_id, type, position, dimensions	Represents openings placed within walls.
Furniture	furniture_id, plan_id, item_type, position, dimensions	Represents furniture and interior objects.

Table 4.5: Proposed Logical Entities

4.7 SIX-SPRINT PLAN

The project is planned across six sprints, with each sprint covering approximately two weeks. Relative week numbers are used because the project start date has not been specified.

Sprint	Schedule	Focus	Hours	Expected Increment
Sprint 1	Weeks 1–2	Interface, floor plan creation, navigation, responsive foundation	17	Initial floor plan editor and interface.
Sprint 2	Weeks 3–4	Wall editing, doors, and windows	11	Floor plan with essential architectural elements.
Sprint 3	Weeks 5–6	Furniture arrangement and floor plan modification	13	Editable layout with interior elements.
Sprint 4	Weeks 7–8	3D model generation, rotation, and zoom	14	Interactive 3D floor plan viewer.
Sprint 5	Weeks 9–10	Optional save/load functionality	6	Floor plan persistence, if included in scope.
Sprint 6	Weeks 11–12	Rendering optimization, testing, integration, and documentation	5	Reviewed release candidate and final documentation.
Total	12 weeks	All 12 user stories	66	Planned project release

Table 4.6: Six-Sprint Release Plan

4.8 UI/UX WIREFRAME PLAN

The UI/UX plan focuses on making the floor planning process understandable and accessible. The interface should provide a clear workspace for creating the floor plan, tools for placing architectural elements, and a separate or integrated viewer for examining the 3D model.

Table 4.8: UI/UX Design Plan

Interface Component	Design Objective
Header and navigation	Provide access to project controls and view options.
Tool panel	Group tools for walls, doors, windows, rooms, and furniture.
Floor plan canvas	Allow users to create and edit the two-dimensional layout.
Properties panel	Display and modify selected object dimensions and properties.
3D preview	Show the generated model and provide rotation and zoom controls.
Feedback messages	Inform users about editing actions, errors, and save/load results where supported.

The wireframe is illustrative and should be adjusted to match the implemented application. The final report should include actual screenshots of the design interface, floor plan editing workspace, 3D visualization, and responsive layout where available.

4.9 TECHNOLOGY STACK

The technology stack should support interactive browser-based floor planning, architectural element manipulation, three-dimensional visualization, and maintainable development.

Layer	Suggested Technology	Purpose
Frontend	HTML5, CSS3, JavaScript or React.js	Develop the interactive user interface.
UI styling	CSS or responsive CSS framework	Maintain layout consistency and responsive design.
3D visualization	Three.js or another suitable 3D rendering library	Render and navigate three-dimensional models.
Application logic	JavaScript or a suitable backend technology, depending on requirements	Process floor plan editing and application operations.
Data storage	Browser storage or a database, depending on persistence requirements	Store and retrieve floor plans when supported.

Table 4.7: Proposed Technology Stack

The table presents the technology stack identified for the proposed system. React, TypeScript, and Vite support the frontend implementation, while Three.js provides three-dimensional visualization capabilities. Node.js and Express.js support backend operations, and MongoDB with Mongoose provides data persistence for user accounts and saved floor plans.

The system is designed to integrate these technologies to provide a unified platform for creating, modifying, visualizing, saving, and sharing house floor plans. Additional capabilities, such as blueprint import, plan export, messaging, and engineer review, depend on the corresponding modules being implemented and configured.

CHAPTER 5. PROJECT PROGRESS AND MANAGEMENT

The 3D House Floor Plan Designer Project follows an incremental development approach in which the system is planned, developed, tested, and reviewed across six two-week sprints over a proposed period of twelve weeks. Each sprint focuses on a specific set of requirements, such as project setup, 2D floor plan editing, architectural elements, 3D visualization, project storage, and final testing. Project progress is monitored using a sprint board, Daily Scrum records, sprint reviews, and retrospectives. These activities help the development team track completed work, identify technical difficulties, coordinate responsibilities, collect feedback, and improve the development process.

5.1 SPRINT 1 – REQUIREMENTS AND PROJECT SETUP

Sprint duration: Weeks 1–2 (10 working days, excluding holidays)

Sprint objective: Establish the project foundation, identify functional requirements, prepare the development environment, create the initial application structure, and configure source-code management.

Planned activities:

- Identify user roles and functional requirements.
- Prepare the frontend and backend project structure.
- Configure React, TypeScript, Vite, Node.js, and Express.js as required.
- Establish the initial floor plan editor interface.
- Configure the Git repository and organize source files.

Planned increment: An initial application structure with a basic interface, project setup, and source-code repository.

5.1.1 SPRINT 1 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Status
US01	User Registration and Login	Basil Eldho	To Do
US02	Dashboard	Gino Peter	In Progress
US03	Project Creation	Jesil C J	To Do
US22	Testing and Deployment Setup	Entire team	To Do

Table 5.1: Sprint 1 – Scrum Board

Task ID	Activity	Responsibility	Expected evidence
T1.1	Confirm requirements and acceptance criteria	Entire team	Requirements document
T1.2	Configure frontend project structure	Gino Peter and Jesil C J	Initial interface source files
T1.3	Prepare backend structure and API foundation	Basil Eldho	Backend source files
T1.4	Create GitHub repository and record changes	Entire team	Repository and commit history
T1.5	Perform initial setup and integration checks	Entire team	Setup checklist and test notes

Table 5.2: Sprint 1 Task-Level Work Plan

Definition of Done for Sprint 1

- Initial project requirements are documented.
- The development environment is configured.
- The basic application structure is available.
- Frontend and backend components are organized.
- Source code is tracked in GitHub.
- Initial setup checks are completed and any blocking issues are recorded.

5.1.2 DAILY SCRUM LOG

Day	Progress to review	Possible impediment	Action agreed
Day 1	Discuss project scope and user requirements	Some requirements may need clarification	Confirm core features and user roles
Day 2	Prepare the development environment	Package or configuration issues	Verify dependencies and setup
Day 3	Establish frontend project structure	Folder organization may be unclear	Agree on component and file structure
Day 4	Prepare backend structure	API responsibilities may overlap	Define backend modules
Day 5	Draft the initial application interface	Layout requirements may change	Review interface with the team

Day 6	Begin project creation functionality	Data fields need agreement	Define project dimensions and units
Day 7	Configure source control	Merge or commit inconsistencies	Use descriptive commit messages
Day 8	Integrate available frontend and backend work	Interface/API mismatches	Verify request and response formats
Day 9	Check initial application setup	Missing documentation or test evidence	Record issues and setup results
Day 10	Review sprint deliverables	Some tasks may remain incomplete	Update the board and prepare Sprint 2

Table 5.3: Sprint 1 – Daily Scrum Log

This is an illustrative log. Replace each entry with actual discussions, blockers, and actions from your team's meetings.

5.1.3 SPRINT REVIEW / DEMO

Item	**Planned review**
Sprint goal	Establish the initial project foundation
Features demonstrated	Application structure, initial interface, development setup
Demonstration method	Browser demonstration and source-code review
Verification	Project setup, repository access, and basic application execution
Feedback to record	Interface changes, missing requirements, setup issues
Follow-up action	Resolve identified issues and confirm the next sprint backlog
Evidence to retain	Screenshots, repository URL, commit history, review notes

Table 5.4: Sprint 1 – Review and Demonstration Record

Demonstration sequence

1. Explain the objectives and requirements of the 3D House Floor Plan Designer.
2. Show the initial application interface.
3. Explain the frontend and backend project structure.
4. Demonstrate the configured GitHub repository.
5. Record feedback and identify improvements for Sprint 2.

5.1.4 SPRINT RETROSPECTIVE

Area	Observation to evaluate	Improvement action	Responsibility
Requirements	Some feature details may need clarification	Finalize acceptance criteria before implementation	Entire team
Frontend setup	Component organization may need refinement	Maintain a consistent project structure	Gino Peter and Jesil C J
Backend setup	API responsibilities may overlap	Define and document backend modules	Basil Eldho
Version control	Changes may need better documentation	Maintain descriptive commits	Entire team
Collaboration	Integration tasks require coordination	Review progress and dependencies regularly	Entire team
Testing	Setup evidence may be incomplete	Maintain a test and evidence checklist	Entire team

Table 5.5: Sprint 1 – Retrospective Record

What went well: Evaluate whether the project scope, development environment, initial structure, and source-control workflow were established.

What could be improved: Identify setup delays, unclear requirements, integration difficulties, or gaps in task allocation.

Action items for Sprint 2:

1. Finalize the initial project creation workflow.
2. Develop the 2D floor plan editor foundation.
3. Establish consistent component and geometry-management conventions.
4. Continue maintaining GitHub commits and project documentation.
5. Verify each completed task against its acceptance criteria.

5.2 SPRINT 2 – 2D FLOOR PLAN EDITOR

Sprint duration: Weeks 3–4 (10 working days, excluding holidays)

Sprint objective: Develop the core 2D floor plan editing functionality, enabling users to create house projects and arrange rooms, walls, doors, windows, stairs, and dimensions.

Planned user stories: US03 – Project Creation; US04 – 2D Floor Plan Editor; US05 – Room and Wall Management; US06 – Doors, Windows and Stairs; US07 – Dimension and Unit Management.

Planned increment: A basic 2D floor plan editor that supports project dimensions and the placement and editing of essential architectural elements.

5.2.1 SPRINT 2 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Illustrative status
US04	2D Floor Plan Editor	Gino Peter	In Progress
US05	Room and Wall Management	Jesil C J	To Do
US06	Doors, Windows and Stairs	Gino Peter	To Do
US07	Dimension and Unit Management	Basil Eldho	To Do

Table 5.6: Sprint 2 – Scrum Board

5.2.2 DAILY SCRUM LOG

Day	Progress to review	Possible impediment	Action agreed
Day 1	Review editor requirements	Editing behaviour needs clarification	Confirm editor functionality
Day 2	Develop the editor layout	Canvas arrangement issues	Refine the editing area
Day 3	Implement wall placement	Wall coordinates may be inaccurate	Verify geometry calculations
Day 4	Implement room management	Room boundaries may overlap	Add boundary checks
Day 5	Integrate room and wall editing	Integration inconsistencies	Test the combined editor
Day 6	Add door and window placement	Opening alignment issues	Validate placement against walls
Day 7	Add stairs and other elements	Element positioning issues	Standardize placement rules
Day 8	Implement dimensions and units	Conversion inconsistencies	Verify unit calculations
Day 9	Test editing operations	Editing defects	Correct and retest issues
Day 10	Review the editor increment	Incomplete acceptance criteria	Update the board and review results

Table 5.7: Sprint 2 – Daily Scrum Log

5.2.3 SPRINT REVIEW / DEMO

Item	Planned review
Sprint goal	Develop the core 2D floor plan editor
Features demonstrated	Room and wall placement, openings, stairs, dimensions
Demonstration method	Live editor demonstration
Verification	Placement, editing, dimensions, and unit checks
Feedback to record	Editor usability and geometry corrections
Follow-up action	Refine the editor and prepare 3D visualization
Evidence to retain	Editor screenshots, test results, review notes

Table 5.8: Sprint 2 – Review and Demonstration Record

Demonstration sequence

1. Create or open a house project.
2. Set the plot dimensions and measurement units.
3. Add rooms and walls to the floor plan.
4. Place doors, windows, and stairs where supported.
5. Verify dimensions and review the resulting 2D plan.

5.2.4 SPRINT RETROSPECTIVE

Area	Observation to evaluate	Improvement action	Responsibility
Editor usability	Some editing actions may require clarification	Improve controls and feedback	Gino Peter and Jesil C J
Geometry	Placement calculations may produce errors	Add geometry validation	Entire team
Integration	Components may use inconsistent data structures	Agree on shared data formats	Entire team
Testing	Edge cases may be missed	Test different dimensions and layouts	Entire team

Action items for Sprint 3:

1. Integrate Three.js for 3D visualization.
2. Convert the 2D floor plan data into a 3D representation.
3. Implement camera rotation and zoom.
4. Begin furniture placement and appearance customization.

5.3 SPRINT 3 – 3D VISUALIZATION AND INTERIOR DESIGN

Sprint duration: Weeks 5–6 (10 working days, excluding holidays)

Sprint objective: Implement 3D visualization and improve the presentation of house designs through interactive viewing, furniture placement, and appearance customization.

Planned user stories: US08 – 3D Visualization; US09 – Furniture Placement; US10 – Materials and Appearance.

Planned increment: A 3D house model generated from the floor plan, with interactive viewing and supported interior design elements.

5.3.1 SPRINT 3 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Illustrative status
US08	3D Visualization	Gino Peter	In Progress
US09	Furniture Placement	Jesil C J	To Do
US10	Materials and Appearance	Jesil C J	To Do
—	3D and editor integration	Basil Eldho and team	To Do

5.3.2 DAILY SCRUM LOG

Day	Progress to review	Possible impediment	Action agreed
Day 1	Review 3D visualization requirements	Mapping 2D data to 3D needs clarification	Define model-generation rules
Day 2	Configure Three.js scene	Scene configuration issues	Verify camera, lighting, and renderer
Day 3	Generate walls and floors in 3D	Incorrect geometry	Check model dimensions
Day 4	Integrate the model with the editor	Data synchronization issues	Verify shared floor plan data
Day 5	Add rotation and zoom controls	Difficult camera navigation	Refine camera controls
Day 6	Add furniture objects	Placement and scaling issues	Standardize object dimensions
Day 7	Implement materials and appearance	Material inconsistencies	Test appearance settings
Day 8	Check rendering performance	Complex models may render slowly	Review scene complexity

Day 9	Test different floor plans	Some configurations may fail	Test representative layouts
Day 10	Demonstrate the 3D increment	Remaining rendering defects	Record findings and prioritize fixes

5.3.3 SPRINT REVIEW / DEMO

Item	**Planned review**
Sprint goal	Provide an interactive 3D representation
Features demonstrated	3D model, rotation, zoom, furniture, appearance options
Demonstration method	Live 3D viewer demonstration
Verification	Model dimensions, camera controls, object placement
Feedback to record	Visualization quality and navigation usability
Follow-up action	Improve rendering and begin project persistence
Evidence to retain	3D screenshots, demonstration notes, test results

Demonstration sequence

1. Open a completed 2D floor plan.
2. Generate or display its 3D representation.
3. Rotate and zoom the model.
4. Demonstrate furniture placement where implemented.
5. Show available materials or appearance options.
6. Record feedback on model accuracy and usability.

5.3.4 SPRINT RETROSPECTIVE

Area	**Observation to evaluate**	**Improvement action**	**Responsibility**
Visualization	The 3D model may not match the 2D plan	Validate dimensions and geometry	Gino Peter and Jesil C J
Rendering	Large models may affect performance	Optimize objects and rendering	Entire team
Furniture	Objects may require scaling adjustments	Establish placement and size rules	Jesil C J
Integration	Editor and viewer data may become inconsistent	Test updates between views	Entire team

What went well: Evaluate whether the team achieved a usable 3D visualization increment.

What could be improved: Identify rendering performance issues, dimension mismatches, and object-placement difficulties.

Action items for Sprint 4:

1. Implement saving and reopening floor plan projects.
2. Develop project-sharing functionality.
3. Investigate supported blueprint-import functionality.
4. Test the consistency of saved and reopened designs.

5.4 SPRINT 4 – PROJECT STORAGE, SHARING AND BLUEPRINT IMPORT

Sprint duration: Weeks 7–8 (10 working days, excluding holidays)

Sprint objective: Enable users to preserve their floor plan work, reopen saved projects, share designs, and use supported blueprint-import functionality.

Planned user stories: US11 – Blueprint Import; US12 – Save and Load Project; US13 – Share Project; US14 – Guest or Shared Viewer.

Planned increment: A project-management workflow that supports saving, retrieving, and sharing designs, with blueprint import and guest viewing where implemented.

5.4.1 SPRINT 4 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Illustrative status
US11	Blueprint Import	Gino Peter	To Do
US12	Save and Load Project	Basil Eldho	In Progress
US13	Share Project	Basil Eldho	To Do
US14	Guest or Shared Viewer	Jesil C J	To Do

5.4.2 Daily Scrum Log

Day	Progress to review	Possible impediment	Action agreed
Day 1	Review project-storage requirements	Project data format may need refinement	Define the saved project structure
Day 2	Develop save-project operations	API integration issues	Test save requests
Day 3	Develop load-project operations	Some design properties may be missing	Verify stored geometry and dimensions
Day 4	Integrate persistence with the editor	Unsaved changes may be lost	Test save and reopen workflows

Day 5	Develop project-sharing operations	Access permissions need clarification	Define sharing rules
Day 6	Prepare shared-viewer functionality	Unauthorized access concerns	Verify access restrictions
Day 7	Investigate blueprint import	Unsupported file formats	Define supported input formats
Day 8	Test project recovery and sharing	Inconsistent saved data	Retest complete workflows
Day 9	Review project persistence	Missing validation cases	Test invalid and incomplete data
Day 10	Demonstrate the increment	Some functions may remain incomplete	Document outcomes and pending work

5.4.3 Sprint Review / Demo

Item	**Planned review**
Sprint goal	Improve project persistence and sharing
Features demonstrated	Save, reopen, share, shared viewing, blueprint import if implemented
Demonstration method	Application demonstration and database/API checks
Verification	Data consistency and access permissions
Feedback to record	Save workflow, sharing usability, import limitations
Follow-up action	Resolve data issues and prepare engineer workflows
Evidence to retain	Screenshots, test cases, API results, review notes

Demonstration sequence

1. Create and save a floor plan.
2. Reopen the project and check its dimensions and elements.
3. Demonstrate the project-sharing workflow.
4. Open a shared design using the supported viewer.
5. Demonstrate blueprint import if the feature is available.
6. Record any data loss, access, or import issues.

5.4.4 SPRINT RETROSPECTIVE

Area	Observation to evaluate	Improvement action	Responsibility
Persistence	Saved projects may lose some properties	Validate all required project fields	Basil Eldho
Sharing	Permissions may be unclear	Test access rules	Basil Eldho
Blueprint import	Imported plans may not be geometrically accurate	Validate imported references	Entire team
Integration	Database and frontend errors may be difficult to trace	Improve error reporting	Entire team

What went well: Evaluate the reliability of project saving, reopening, and sharing. What could be improved: Identify data-consistency issues, access-control gaps, and blueprint-import limitations.

Action items for Sprint 5:

1. Develop engineer plan-review and plan-creation workflows.
2. Implement supported plan-export functionality.
3. Develop messaging and notifications if included in the implementation.
4. Test project sharing and engineer-related operations.

5.5 SPRINT 5 – ENGINEER WORKFLOWS, EXPORT AND COLLABORATION

Sprint duration: Weeks 9–10 (10 working days, excluding holidays)

Sprint objective: Improve the system's professional workflows by supporting engineer plan review, plan creation, plan sharing, and design export. Messaging and notifications are included where implemented.

Planned user stories: US15 – Engineer Plan Review; US16 – Engineer Plan Creation; US17 – Engineer Plan Sharing; US18 – PDF or Image Export; US19 – Chat and Notifications.

Planned increment: A more complete design-management workflow that supports engineer-related activities and the export of floor plan designs in supported formats.

5.5.1 SPRINT 5 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Illustrative status
US15	Engineer Plan Review	Basil Eldho	In Progress
US16	Engineer Plan Creation	Gino Peter	To Do
US17	Engineer Plan Sharing	Basil Eldho	To Do
US18	PDF or Image Export	Jesil C J	To Do
US19	Chat and Notifications	Basil Eldho and team	To Do

5.5.2 DAILY SCRUM LOG

Day	Progress to review	Possible impediment	Action agreed
Day 1	Review engineer workflow requirements	User permissions may need clarification	Define permitted engineer actions
Day 2	Develop plan-review operations	Review status may be unclear	Define review outcomes
Day 3	Develop engineer plan creation	Existing editor components may need adaptation	Reuse validated editor components
Day 4	Integrate engineer plan sharing	Permission checks may fail	Test sharing access
Day 5	Implement plan export	Output formatting issues	Verify exported dimensions and layout
Day 6	Test supported export formats	Some objects may be omitted	Compare exports with the original plan
Day 7	Develop messaging if included	Message storage or delivery issues	Test message handling
Day 8	Integrate notifications if included	Events may not trigger correctly	Verify notification conditions
Day 9	Conduct workflow integration tests	Cross-module defects	Test complete user journeys
Day 10	Demonstrate available features	Some optional functions may be incomplete	Record results and pending tasks

5.5.3 SPRINT REVIEW / DEMO

Item	Planned review
Sprint goal	Support engineer workflows and plan export
Features demonstrated	Plan review, engineer plan creation, sharing, export
Demonstration method	End-to-end application demonstration
Verification	Workflow completion, permissions, exported output
Feedback to record	Engineer usability and export quality
Follow-up action	Resolve defects and prepare final testing
Evidence to retain	Exported files, screenshots, test records, review notes

Demonstration sequence

1. Open a shared or submitted floor plan.
2. Demonstrate engineer review functionality if implemented.
3. Create or modify a plan using the available editor.
4. Share the design using the supported workflow.
5. Export the plan to a supported image or PDF format.
6. Demonstrate messaging or notifications if implemented.

5.5.4 SPRINT RETROSPECTIVE

Area	Observation to evaluate	Improvement action	Responsibility
Engineer workflow	Review actions may need clearer status information	Improve workflow feedback	Basil Eldho
Export	Output may differ from the editor	Verify export dimensions and completeness	Gino Peter and Jesil C J
Permissions	Sharing rules may need further testing	Test authorized and unauthorized access	Basil Eldho
Integration	Multiple modules may introduce defects	Conduct end-to-end tests	Entire team

What went well: Evaluate whether engineer workflows and export capabilities operate as intended. What could be improved: Identify workflow errors, export inconsistencies, and incomplete integration.

Action items for Sprint 6:

1. Complete remaining high-priority functionality.
2. Perform system and regression testing.
3. Verify project persistence, dimensions, and geometry.
4. Resolve critical defects and prepare the final documentation.

5.6 SPRINT 6 – FINAL TESTING, DEPLOYMENT AND DOCUMENTATION

Sprint duration: Weeks 11–12 (10 working days, excluding holidays)

Sprint objective: Validate the integrated application, complete remaining administrative and validation features where planned, resolve important defects, prepare deployment, and finalize the project documentation.

Planned user stories: US20 – Administrator Management; US21 – Validation and Error Handling; US22 – Testing and Deployment Setup.

Planned increment: A tested and documented release candidate of the 3D House Floor Plan Designer, containing the functionality completed during the preceding sprints.

5.6.1 SPRINT 6 BOARD – POPULATED SAMPLE

ID	Task / User Story	Owner	Illustrative status
US20	Administrator Management	Basil Eldho	To Do
US21	Validation and Error Handling	Gino Peter and Jesil C J	In Progress
US22	Testing and Deployment Setup	Entire team	To Do
—	Final documentation and demonstration	Entire team	To Do

5.6.2 DAILY SCRUM LOG

Day	Progress to review	Possible impediment	Action agreed
Day 1	Review outstanding requirements	Incomplete tasks may remain	Prioritize essential work
Day 2	Test registration and project workflows	Authentication or data errors	Correct and retest defects
Day 3	Test floor plan geometry and dimensions	Invalid dimensions or element placement	Strengthen validation
Day 4	Test 3D visualization	Model-rendering inconsistencies	Correct visualization defects
Day 5	Test save, load, and sharing	Data or permission errors	Verify complete workflows
Day 6	Test export and optional features	Missing output elements	Correct supported export issues
Day 7	Conduct regression testing	Earlier fixes may affect other features	Repeat relevant test cases
Day 8	Prepare deployment configuration	Environment differences	Verify configuration and dependencies
Day 9	Complete documentation and screenshots	Missing evidence	Organize final project records
Day 10	Conduct final review and demonstration	Remaining unresolved defects	Document release status and limitations

5.6.3 SPRINT REVIEW / DEMO

Item	Planned review
Sprint goal	Validate and prepare the final project increment
Features demonstrated	Completed floor plan editing, 3D viewing, storage, and other implemented features
Demonstration method	End-to-end application demonstration
Verification	Functional tests, geometry checks, persistence, and error handling
Feedback to record	Remaining defects and recommended improvements
Follow-up action	Resolve critical issues and record known limitations
Evidence to retain	Test report, screenshots, repository details, final documentation

Demonstration sequence

1. Launch the application in the configured environment.
2. Create a house project and specify its dimensions.
3. Create or edit a 2D floor plan.
4. View the design in 3D.
5. Save and reopen the project if persistence is implemented.
6. Demonstrate sharing, export, or engineer workflows that are available.
7. Explain testing results, limitations, and future enhancements.

5.6.4 SPRINT RETROSPECTIVE

Area	Observation to evaluate	Improvement action	Responsibility
Functional testing	Some workflows may remain incomplete	Resolve critical defects before submission	Entire team
Geometry validation	Invalid layouts may cause inconsistent models	Validate dimensions and placement	Gino Peter and Jesil C J
Backend reliability	API or database failures may affect workflows	Test error handling and persistence	Basil Eldho
Deployment	Configuration may differ across environments	Document setup and deployment steps	Entire team
Documentation	Screenshots or test records may be missing	Verify all report evidence	Entire team
Team coordination	Final tasks may overlap	Assign clear ownership and deadlines	Entire team

What went well: Evaluate the integration of the application, completion of testing, and preparation of project documentation.

What could be improved: Record remaining defects, incomplete features, deployment difficulties, and improvements for future versions.

Final action items

1. Resolve critical defects that prevent core functionality.
2. Verify all implemented features against their acceptance criteria.
3. Record incomplete or optional functionality honestly.
4. Organize the final source code, screenshots, and testing evidence.
5. Prepare the final demonstration and project report.

5.7 OVERALL PROJECT PROGRESS SUMMARY

Sprint	Main focus	Expected deliverable
Sprint 1	Requirements and project setup	Initial application structure and source-control setup
Sprint 2	2D floor plan editor	Rooms, walls, openings, stairs, and dimensions
Sprint 3	3D visualization and interior design	Interactive 3D model and supported furniture features
Sprint 4	Storage, sharing, and blueprint import	Save/load workflows and supported sharing functions
Sprint 5	Engineer workflows and export	Plan review, sharing, and export functionality
Sprint 6	Testing, deployment, and documentation	Tested release candidate and final project report

The sprint plan provides a structured approach to developing the 3D House Floor Plan Designer incrementally. Each sprint builds on the previous increment, allowing the team to verify functionality and address issues before proceeding to later stages. Actual progress should be measured using accepted user stories, completed tasks, test results, and documented sprint outcomes. Planned deliverables must not be reported as completed unless the relevant features have been implemented and verified.

CHAPTER 6. BURNDOWN MONITORING

6.1 INTRODUCTION

Burndown monitoring is an Agile project management technique used to track the amount of work remaining against the time available. In the 3D House Floor Plan Designer Project, burndown monitoring is used at two levels: sprint burndown, which tracks the remaining work within an individual sprint, and product burndown, which tracks the remaining workload across all six sprints.

The project is planned for six two-week sprints over twelve weeks. The work includes project setup, 2D floor plan editing, room and wall management, 3D visualization, furniture placement, project saving and sharing, engineer workflows, testing, and deployment.

For consistency, this chapter uses estimated work hours as the primary burndown unit. Story points are maintained separately in the product backlog and should not be converted directly into hours without a defined estimation method. The burndown charts help the team compare planned and actual progress, identify delays, investigate impediments, and take corrective action when the work does not proceed as expected.

6.2 SPRINT BURNDOWN

A sprint burndown chart shows the estimated hours of work remaining during a particular sprint. At the start of a sprint, the chart begins with the sprint's planned workload. As tasks are completed and verified, the remaining work decreases toward zero.

The following formula is used:

Remaining Work=Sprint Baseline Hours-Completed Work Hours

The team records the estimated remaining work at regular intervals during each sprint. The ideal burndown line represents the planned reduction in work, while the actual line represents the remaining effort recorded by the team.

Sprint	Duration	Planned focus	Baseline hours
Sprint 1	Weeks 1–2	Requirements, project setup, initial interface and source control	15
Sprint 2	Weeks 3–4	2D editor, rooms, walls, openings and dimensions	20
Sprint 3	Weeks 5–6	3D visualization, furniture and appearance	18
Sprint 4	Weeks 7–8	Blueprint import, saving, loading and sharing	17
Sprint 5	Weeks 9–10	Engineer workflows, plan export and collaboration	15
Sprint 6	Weeks 11–12	Testing, validation, deployment and documentation	15
Total	12 weeks	Complete planned project scope	100

The 100-hour baseline above is a proposed planning estimate for organizing the six sprints, not a verified total from actual work logs. It includes implementation, integration, testing, and documentation activities. The team should confirm the allocation against its detailed task estimates and revise it if necessary.

6.3 SPRINT-WISE BURNDOWN PLAN

The following tables illustrate how remaining work can be monitored during each sprint. Each sprint is assumed to contain ten working days, excluding holidays. The ideal remaining effort decreases linearly, while the sample actual values illustrate possible progress patterns.

6.3.1 SPRINT 1 – REQUIREMENTS AND PROJECT SETUP

Baseline: 15 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	15	15
2	12	13
4	9	10
6	6	7
8	3	3
10	0	0

Interpretation: The sample shows slower progress during initial project setup, followed by faster completion toward the end of the sprint. The team should verify the requirements, initial application structure, development environment, and GitHub workflow before accepting the increment.

6.3.2 SPRINT 2 – 2D FLOOR PLAN EDITOR

Baseline: 20 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	20	20
2	16	18
4	12	14
6	8	9
8	4	4
10	0	0

Interpretation: The sample indicates an early delay while developing the 2D editor and handling geometry. The team should check room and wall placement, door and window positioning, stairs, dimensions, and unit conversion before accepting the sprint increment.

6.3.3 SPRINT 3 – 3D VISUALIZATION AND INTERIOR DESIGN

Baseline: 18 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	18	18
2	14.4	15
4	10.8	12
6	7.2	8
8	3.6	4
10	0	0

Interpretation: The sample illustrates gradual progress toward an interactive 3D model. The team should verify that the Three.js model corresponds to the 2D floor plan and that camera rotation, zoom, furniture placement, and appearance options work as intended where implemented.

6.3.4 SPRINT 4 – PROJECT STORAGE AND SHARING

Baseline: 17 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	17	17
2	13.6	15
4	10.2	12
6	6.8	8
8	3.4	4
10	0	0

Interpretation: The sample shows a delay during project persistence and sharing work. The team should verify that saved floor plans preserve geometry, dimensions, and objects, and that users can reopen and share projects according to the implemented permission rules. Blueprint import should be tested if included.

6.3.5 SPRINT 5 – ENGINEER WORKFLOWS AND PLAN EXPORT

Baseline: 15 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	15	15
2	12	13
4	9	10

6	6	7
8	3	3
10	0	0

Interpretation: The sample indicates a small deviation from the ideal plan during engineer workflow and export implementation. The team should test plan review, engineer plan creation, sharing, and supported PDF or image export. Messaging and notifications should be included in testing only if implemented.

6.3.6 SPRINT 6 – TESTING, DEPLOYMENT AND DOCUMENTATION

Baseline: 15 hours

Day	Ideal remaining (hrs)	Sample actual remaining (hrs)
0	15	15
2	12	14
4	9	11
6	6	7
8	3	4
10	0	0

Interpretation: The sample illustrates slower progress during final verification, followed by completion of the remaining work. Before the final release, the team should complete regression testing, verify project storage and geometry, address critical defects, check deployment configuration, and organize the final documentation.

Tables 6.2–6.7: Illustrative sprint burndown checkpoints. The actual values are examples for documentation purposes. They should be replaced with the team's recorded remaining-work estimates from daily Scrum meetings, sprint boards, or work logs.

6.4 PRODUCT BURNDOWN

The product burndown chart monitors the remaining estimated effort for the entire 3D House Floor Plan Designer Project across all six sprints. Unlike a sprint burndown chart, which resets at the beginning of each sprint, the product burndown maintains a continuous view of the overall project workload.

The initial proposed product baseline is 100 hours. At the end of each sprint, the team updates the remaining product effort based on completed work and any approved changes to the scope or estimates.

Product Work Remaining = 100 - Cumulative Completed Work

Milestone	Sprint completed	Planned cumulative work (hrs)	Planned remaining (hrs)	Illustrative actual remaining (hrs)
Project start	—	0	100	100
End of Sprint 1	1	15	85	85
End of Sprint 2	2	35	65	68
End of Sprint 3	3	53	47	50
End of Sprint 4	4	70	30	34
End of Sprint 5	5	85	15	18
End of Sprint 6	6	100	0	0

Table 6.8: Product Burndown Monitoring Across Six Sprints

3D House Floor Plan Designer — Product Burndown

Planned and illustrative remaining effort at each sprint milestone, in hours.

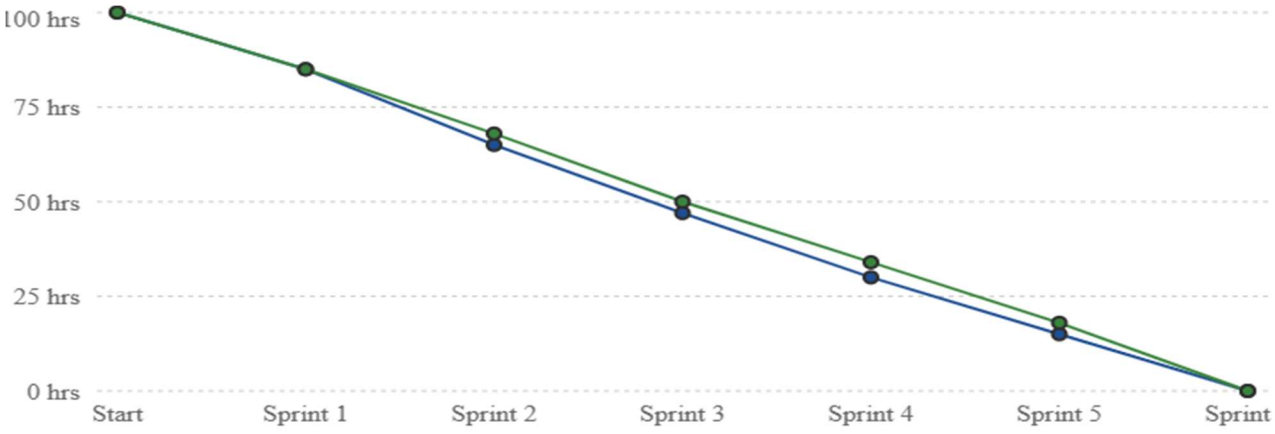


Figure 6.1: Product Burndown Chart Across Six Sprints

■ Planned remaining

■ Illustrative actual remaining

The chart illustrates a planned reduction from 100 hours to zero over twelve weeks. In the example, the illustrative actual line remains above the planned line during Sprints 2–5, suggesting that some work is taking longer than expected or that additional integration and verification work remains. The sample then reaches zero at the end of Sprint 6.

These values are illustrative, not measured project performance. In the final report, actual remaining effort should be calculated from the team's records. If new requirements are added or estimates change, the team should document the baseline revision rather than silently changing the original plan.

6.5 BURNDOWN MONITORING AND CORRECTIVE ACTIONS

The team should review burndown data during daily Scrum meetings and sprint reviews. Deviations between planned and actual remaining work should be investigated to identify their causes and determine suitable corrective actions.

Observed condition	Possible cause	Recommended action
Actual remaining work is above the ideal line	Delayed implementation or underestimated complexity	Identify blockers and revise the immediate work plan
Remaining work does not decrease for several days	Tasks are blocked or too large	Break down tasks and resolve dependencies
Remaining work decreases sharply near sprint end	Work updates are delayed or tasks are completed in batches	Update the board regularly and apply clear completion criteria
Work remains at sprint end	Incomplete stories or unplanned work	Review scope and reprioritize remaining items
Product burndown increases	New requirements or revised estimates	Record the baseline change and explain its impact
Work reaches zero but defects remain	Completion criteria are insufficient	Reopen affected work and complete verification
3D visualization takes longer than planned	Geometry, rendering, or integration difficulties	Prioritize core visualization, investigate performance, and reassess estimates
Saved projects contain missing or incorrect data	Persistence or data-validation defects	Test save/load workflows and verify project data
Exported floor plans differ from the editor	Export formatting or geometry issues	Compare exported output against the original design and correct discrepancies

Table 6.9: Burndown Variance and Corrective Actions

A burndown chart should represent the estimated work remaining, not simply the number of tasks marked complete. The team should use a consistent measurement method, update its estimates when the scope changes, and avoid marking a feature complete until its acceptance criteria have been verified.

6.6 SUMMARY

This chapter established a burndown monitoring plan for the 3D House Floor Plan Designer Project using six two-week sprints over a planned period of twelve weeks and a proposed total baseline of 100 hours.

Sprint burndown charts provide a short-term view of the work remaining within each sprint, while the product burndown chart tracks the overall workload across the project. The monitoring plan covers requirements and setup, 2D floor plan editing, 3D visualization, project storage and sharing, engineer workflows, testing, deployment, and documentation. The sample burndown schedules demonstrate how the team can compare planned progress with actual remaining work, identify deviations, investigate impediments, and determine corrective actions. These practices help improve transparency, support sprint planning, and reduce the risk of leaving important tasks incomplete at the end of the project.

For the final report, the team should replace all illustrative actual values with verified records from sprint boards, daily Scrum logs, and work estimates. The final charts should reflect the actual project scope, approved changes, completed work, and remaining effort at each reporting point.

CHAPTER 7. SCRUM CEREMONY (DAILY SCRUM CONSOLIDATION) EVIDENCE REGISTER

7.1 INTRODUCTION

This chapter documents the Scrum ceremonies planned and conducted during the development of the **3D House Floor Plan Designer Project**. The project follows an iterative development approach in which the application is developed through six proposed two-week sprints over a twelve-week period. Scrum ceremonies support coordination among team members, monitoring of development progress, evaluation of completed functionality, and continuous improvement throughout the project lifecycle.

The Daily Scrum helps the development team inspect progress toward the Sprint Goal, identify impediments, and agree on immediate actions. Sprint Reviews provide opportunities to demonstrate completed functionality, such as the 2D floor plan editor, room and wall management, three-dimensional visualization, project storage, and sharing. Sprint Retrospectives help the team identify improvements in communication, implementation practices, testing, and collaboration.

Documentation note: The sprint goals and daily activities presented in this chapter are proposed planning records based on the project's backlog and six-sprint structure. They should not be treated as proof that meetings or development tasks have already been completed. Actual meeting dates, participants, progress updates, decisions, repository links, screenshots, and outcomes must be recorded from the team's genuine project evidence.

7.2 SCRUM CEREMONY FRAMEWORK

Scrum ceremonies provide a systematic approach to planning, monitoring, reviewing, and improving the development of the 3D House Floor Plan Designer. The following table identifies the main ceremonies and the evidence that should be maintained.

Ceremony	Purpose	Evidence to Maintain
Sprint Planning	Define the Sprint Goal and select user stories from the Product Backlog.	Sprint backlog, selected user stories, estimates, and planning notes.
Daily Scrum	Inspect progress, identify impediments, and coordinate the next development activities.	Daily Scrum log, progress updates, impediment records, and action register.
Sprint Review	Demonstrate the completed increment and collect feedback on the implemented functionality.	Application screenshots, demonstration notes, feedback, and acceptance observations.
Sprint Retrospective	Identify successful practices and improvements for the next sprint.	Retrospective notes, improvement actions, and follow-up records.
Backlog Refinement	Clarify requirements, acceptance criteria, dependencies, and priorities for upcoming work.	Updated user stories, acceptance criteria, estimates, and priority changes.

7.3 SPRINT GOALS AND DAILY SCRUM CONSOLIDATION

The project Product Backlog contains 22 user stories covering authentication, project creation, floor plan editing, three-dimensional visualization, furniture placement, project persistence, sharing, engineer workflows, exporting, validation, testing, and deployment. The following proposed sprint allocation organizes these stories into six development stages.

Sprint	Sprint Goal	User Stories	Expected Increment
Sprint 1	Establish the project foundation and implement the initial application structure.	US01, US02, US03, US22	Registration and login, dashboard, project creation, initial development environment, and source-control setup.
Sprint 2	Develop the core 2D floor plan editing functionality.	US04, US05, US06, US07	Floor plan editor, room and wall management, doors, windows, stairs, dimensions, and unit management.
Sprint 3	Implement three-dimensional visualization and interior design features.	US08, US09, US10	3D visualization, furniture placement, and materials and appearance controls.
Sprint 4	Implement blueprint import, project persistence, and sharing functionality.	US11, US12, US13, US14	Blueprint import, saving and loading projects, project sharing, and guest or shared viewing.
Sprint 5	Develop engineer workflows, plan sharing, export, and collaboration features.	US15, US16, US17, US18, US19	Engineer plan review, plan creation, plan sharing, PDF or image export, and chat or notifications where implemented.
Sprint 6	Complete administration, validation, testing, deployment, and final documentation.	US20, US21, US22	Administrator management, validation and error handling, testing, deployment checks, and final project evidence.

7.4 DAILY SCRUM CONSOLIDATION EVIDENCE REGISTER

The following registers provide ten daily checkpoints for each proposed two-week sprint, assuming five working days per week. For every Daily Scrum, the team should record what was completed since the previous meeting, what will be done next, any impediments, the responsible team member, and the agreed action.

7.4.1 SPRINT 1 – REQUIREMENTS AND PROJECT SETUP

Sprint Goal: Establish the development environment, clarify the project requirements, implement the initial application structure, and prepare the foundation for user authentication, dashboard functionality, and project creation.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Confirm project scope, backlog items, acceptance criteria, and Sprint Goal.	S1-DS01
D2	Configure the React, TypeScript, and Vite development environment and Git repository.	S1-DS02
D3	Establish the initial frontend structure and application navigation.	S1-DS03
D4	Plan the backend structure using Node.js and Express.js.	S1-DS04
D5	Establish the MongoDB and Mongoose configuration where applicable.	S1-DS05
D6	Develop the initial registration and login interface (US01).	S1-DS06
D7	Develop the initial dashboard interface (US02).	S1-DS07
D8	Implement the project creation workflow (US03).	S1-DS08
D9	Verify the initial application flow, configuration, and source-control records.	S1-DS09
D10	Review the sprint increment, document outstanding issues, and prepare Sprint 2 backlog.	S1-DS10

Table 7.3: Sprint 1 Daily Scrum Evidence Register

Expected review evidence: Initial application screenshots, project folder structure, frontend and backend setup records, repository history, and an acceptance checklist for the completed foundation.

7.4.2 SPRINT 2 – 2D FLOOR PLAN EDITOR

Sprint Goal: Develop the core two-dimensional floor plan editor and support the creation and modification of rooms, walls, doors, windows, stairs, dimensions, and measurement units.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Confirm editor requirements, geometry rules, and acceptance criteria (US04).	S2-DS01
D2	Develop the initial 2D floor plan editing canvas and interaction controls (US04).	S2-DS02
D3	Implement room creation and room modification operations (US05).	S2-DS03
D4	Implement wall placement and wall editing operations (US05).	S2-DS04
D5	Add door and window placement functionality (US06).	S2-DS05
D6	Implement stair placement and related editing behaviour (US06).	S2-DS06
D7	Add dimension display and measurement-unit handling (US07).	S2-DS07
D8	Validate geometry, object positioning, and editing interactions.	S2-DS08
D9	Test the main editor workflows and record identified defects.	S2-DS09
D10	Demonstrate the 2D editor increment and prepare Sprint 3 backlog.	S2-DS10

Expected review evidence: Screenshots of the 2D editor, sample floor plans containing rooms and walls, door and window placement evidence, dimension and unit checks, and test results for editing operations.

7.4.3 SPRINT 3 – 3D VISUALIZATION AND INTERIOR DESIGN

Sprint Goal: Provide three-dimensional visualization of the floor plan and introduce furniture placement and material or appearance customization.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Confirm 3D visualization requirements and the relationship between 2D geometry and the 3D scene (US08).	S3-DS01
D2	Set up the Three.js scene, camera, lighting, and rendering workflow.	S3-DS02
D3	Convert supported floor plan geometry into three-dimensional elements (US08).	S3-DS03
D4	Implement or refine the 3D viewing controls and scene navigation (US08).	S3-DS04
D5	Verify that rooms, walls, and openings are represented consistently in the 3D view.	S3-DS05
D6	Develop furniture placement functionality (US09).	S3-DS06
D7	Add supported material and appearance controls (US10).	S3-DS07
D8	Test scene updates after changes to the floor plan and furniture arrangement.	S3-DS08
D9	Check rendering behaviour, geometry consistency, and identified visual defects.	S3-DS09
D10	Demonstrate the 3D visualization increment and prepare Sprint 4 backlog.	S3-DS10

Table 7.5: Sprint 3 Daily Scrum Evidence Register

Expected review evidence: Genuine screenshots of the 3D scene, comparison of the 2D plan with its 3D representation, furniture placement examples, appearance customization evidence, and recorded visualization test results.

7.4.4 SPRINT 4 – BLUEPRINT IMPORT, PROJECT STORAGE AND SHARING

Sprint Goal: Support importing suitable blueprint files, saving and loading floor plan projects, and sharing projects with permitted users or viewers.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Confirm supported blueprint formats, import requirements, and validation rules (US11).	S4-DS01
D2	Implement or refine the blueprint import workflow (US11).	S4-DS02
D3	Verify imported plan data and handle unsupported or invalid inputs.	S4-DS03
D4	Implement project persistence using the configured backend and database (US12).	S4-DS04

D5	Test saving and retrieving floor plan projects (US12).	S4-DS05
D6	Implement project sharing and access permissions (US13).	S4-DS06
D7	Develop the guest or shared-viewer workflow where supported (US14).	S4-DS07
D8	Test project access, sharing permissions, and persistence behaviour.	S4-DS08
D9	Resolve identified import, storage, and sharing defects.	S4-DS09
D10	Demonstrate import, save/load, and sharing functionality and prepare Sprint 5 backlog.	S4-DS10

Table 7.6: Sprint 4 Daily Scrum Evidence Register

Expected review evidence: Screenshots of blueprint import, supported input examples, save/load test records, project-sharing screens, permission checks, and evidence that shared viewers can access only the intended project information.

7.4.5 SPRINT 5 – ENGINEER WORKFLOWS, EXPORT AND COLLABORATION

Sprint Goal: Implement engineer-related plan workflows, support plan sharing and export, and develop collaboration functionality were included in the implementation.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Review engineer workflow requirements and access rules for plan review (US15).	S5-DS01
D2	Implement or refine engineer plan review functionality (US15).	S5-DS02
D3	Develop engineer plan creation functionality (US16).	S5-DS03
D4	Implement engineer plan sharing operations (US17).	S5-DS04
D5	Develop PDF or image export functionality (US18).	S5-DS05
D6	Verify exported output against the original floor plan.	S5-DS06
D7	Implement or refine chat and notification functionality where supported (US19).	S5-DS07
D8	Test relevant access permissions, export behaviour, and collaboration workflows.	S5-DS08
D9	Resolve identified workflow and integration defects.	S5-DS09
D10	Demonstrate the sprint increment and prepare Sprint 6 backlog.	S5-DS10

Table 7.7: Sprint 5 Daily Scrum Evidence Register

Expected review evidence: Engineer workflow screenshots, plan review and creation examples, sharing and permission test results, exported PDF or image samples, and chat or notification evidence if those features have been implemented.

7.4.6 SPRINT 6 – ADMINISTRATION, VALIDATION, TESTING AND FINAL RELEASE

Sprint Goal: Complete administrator management, strengthen validation and error handling, perform regression testing, verify deployment, and finalize project documentation.

Day	Planned Focus / Progress to Inspect	Evidence ID
D1	Review remaining backlog items, open defects, and release acceptance criteria.	S6-DS01
D2	Implement or verify administrator management functionality (US20).	S6-DS02
D3	Test input validation and error-handling behaviour (US21).	S6-DS03
D4	Execute unit and geometry-related tests for supported functionality.	S6-DS04
D5	Test project creation, editing, visualization, saving, loading, and sharing workflows.	S6-DS05
D6	Verify authentication, permissions, and handling of invalid or incomplete data where applicable.	S6-DS06
D7	Verify deployment configuration and the application's available access points.	S6-DS07
D8	Compile test results, defect records, screenshots, and repository evidence.	S6-DS08
D9	Complete the project report, supporting appendices, and evidence review.	S6-DS09
D10	Conduct the final demonstration, review project outcomes, and record retrospective actions.	S6-DS10

Table 7.8: Sprint 6 Daily Scrum Evidence Register

Expected review evidence: Administrator interface evidence where implemented, validation and error-handling test results, regression checklist, deployment verification, final application screenshots, repository history, and the completed project report.

7.5 IMPEDIMENT AND ACTION REGISTER

The impediment registers record technical or coordination issues that may affect sprint progress. The following entries are anticipated risks for this project, rather than confirmed incidents. Actual status, responsible person, resolution date, and supporting evidence should be added when an issue is observed.

Table 7.9: Consolidated Impediment Register

ID	Potential Impediment	Relevant Sprint	Corrective Action	Status / Evidence
IMP01	Unclear requirements or acceptance criteria.	Sprint 1	Clarify user stories and confirm expected behaviour before implementation.	Record actual status and evidence.
IMP02	Frontend and backend configuration or integration difficulties.	Sprint 1	Verify environment configuration, API contracts, and repository setup.	Record actual status and evidence.
IMP03	Inconsistent room or wall geometry in the 2D editor.	Sprint 2	Validate geometry operations and test representative floor plans.	Record actual status and evidence.

IMP04	Incorrect dimensions or unit-conversion results.	Sprint 2	Verify conversion logic and test supported measurement units.	Record actual status and evidence.
IMP05	Differences between the 2D plan and 3D visualization.	Sprint 3	Compare geometry and object placement between both views and correct mapping issues.	Record actual status and evidence.
IMP06	Slow or unstable 3D rendering.	Sprint 3	Investigate scene complexity, rendering behaviour, and unnecessary updates.	Record actual status and evidence.
IMP07	Imported blueprint contains unsupported or invalid data.	Sprint 4	Validate input files, document supported formats, and provide appropriate error feedback.	Record actual status and evidence.
IMP08	Saved projects cannot be retrieved correctly.	Sprint 4	Test persistence, data validation, and save/load workflows.	Record actual status and evidence.
IMP09	Shared projects expose information to unauthorized users.	Sprint 4	Verify authentication and permission checks for shared resources.	Record actual status and evidence.
IMP10	Exported plans differ from the original floor plan.	Sprint 5	Compare exported files with the editor view and correct formatting or geometry issues.	Record actual status and evidence.
IMP11	Engineer or collaboration workflows are incomplete.	Sprint 5	Review requirements, verify dependencies, and prioritize essential workflow behaviour.	Record actual status and evidence.
IMP12	Testing or deployment defects remain near release.	Sprint 6	Prioritize release-blocking defects, rerun regression tests, and verify the deployed application.	Record actual status and evidence.

These entries should be reviewed during Daily Scrum meetings whenever a relevant issue affects progress. Resolved impediments should include the action taken and the evidence used to verify the resolution.

7.6 EVIDENCE REGISTER AND TRACEABILITY

The evidence register links the planned Scrum ceremonies to the development and verification of the 3D House Floor Plan Designer. Each evidence item should have a unique identifier and a corresponding file, repository link, screenshot, meeting record, or test result.

Evidence Category	Suggested Evidence	Naming Convention
Sprint Planning	Sprint Goal, selected user stories, estimates, and planning notes.	S1_Planning, S2_Planning, etc.
Daily Scrum	Meeting record, progress updates, impediments, and agreed actions.	S1-DS01 to S6-DS10

Sprint Board	Board snapshot showing task status at important checkpoints.	S1_Board, S2_Board, etc.
Sprint Review	Demonstration screenshots, review notes, and feedback.	S1_Review, S2_Review, etc.
Sprint Retrospective	Improvement observations and actions for the next sprint.	S1_Retro, S2_Retro, etc.
Floor Plan Editor Testing	Test cases and results for rooms, walls, openings, dimensions, and units.	S2_Editor_Test, etc.
3D Visualization Evidence	Screenshots and checks comparing the 2D plan with the 3D scene.	S3_3D_Evidence, etc.
Storage and Sharing Testing	Save/load results, permission checks, and sharing evidence.	S4_Storage_Sharing_Test, etc.
Engineer and Export Evidence	Workflow screenshots, exported files, and related verification records.	S5_Engineer_Export_Evidence, etc.
Validation and Regression Testing	Test cases, execution results, defect reports, and retest evidence.	S6_Test, S6_Regression, etc.
Source Control	Repository URL, commit history, branches, and pull requests where used.	Project_Git_Evidence
Final Release	Deployment details, final screenshots, acceptance record, and final report.	Project_Final_Release

Table 7.10: Consolidated Scrum Evidence Index

Evidence should be stored in an organized project folder or repository. The final report should identify the location of each supporting item so that reviewers can trace planned requirements to implementation and testing evidence.

Screenshots must be genuine captures of the implemented application. Meeting records should reflect actual discussions, and test results should correspond to tests that were executed. If a planned feature has not been implemented, it should be marked as pending or not implemented rather than supported by fabricated evidence.

CHAPTER 8. GITHUB / VERSION CONTROL

8.1 VERSION CONTROL AND GIT

In the development of the **3D House Floor Plan Designer Project**, version control and Git play an important role in managing source code, tracking modifications, and supporting collaboration among team members. The project uses technologies such as React, TypeScript, and Vite for frontend development, Three.js for 3D visualization, Node.js and Express.js for backend services, and MongoDB with Mongoose for data management.

Since the application includes several interconnected modules, including the 2D floor plan editor, room and wall management, 3D visualization, project storage, sharing, and export functionality, version control helps maintain consistency and reduces the risk of losing important changes. Git provides a system for tracking source code history, while GitHub can be used to host the remote repository and coordinate development activities throughout the six planned Scrum sprints.

8.1.1 IMPORTANCE OF VERSION CONTROL AND GIT

• Collaboration and Coordination

Git enables multiple developers to work on different parts of the application. The development team consists of Basil Eldho, Gino Peter, and Jesil C J. Basil Eldho is responsible for backend development, idea pitching, and documentation, while Gino Peter and Jesil C J contribute to frontend development, designing, and documentation. Separate branches can be used for tasks such as backend APIs, floor plan editing, and 3D visualization, helping the team coordinate changes before integration.

• Tracking Changes and History

Git maintains a history of committed modifications, including the author, date, and commit message. This helps the team track the development of features such as room placement, wall editing, dimension management, and Three.js visualization. The commit history can also support debugging and provide evidence of incremental development during Scrum reviews.

• Reverting Changes and Error Recovery

If a code modification introduces an error, Git allows developers to inspect previous versions and restore or reverse changes when necessary. This is particularly useful when modifying geometry calculations, project-saving operations, authentication, or 3D rendering, where incorrect changes may affect existing functionality.

• Improving Code Quality

GitHub can support code review through branches and pull requests when these practices are used. Team members can review changes related to floor plan editing, database operations, validation, and project sharing before integration. This helps identify potential defects and encourages maintainable code.

• Backup and Recovery

A remote GitHub repository provides a backup of committed source code. If local project files are lost, the committed files can be recovered from the remote repository. Developers should commit and push changes regularly because uncommitted local work is not automatically backed up.

• Supporting Scrum Development

Git and GitHub can help connect code changes with the project's user stories and sprint goals. For example, Sprint 2 focuses on the 2D floor plan editor and room management, while Sprint 3 focuses on 3D visualization and furniture placement. Repository history can help track implementation progress and support Sprint Reviews and Retrospectives.

8.2 GITHUB REPOSITORY AND DEVELOPMENT WORKFLOW

The GitHub repository can contain the frontend, backend, testing resources, configuration files, and project documentation. A possible repository structure is shown below. The actual structure should match the implemented project.

```
3D-House-Floor-Plan-Designer/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── editor/
│   │   └── visualization/
│   └── package.json
├── backend/
│   ├── src/
│   │   ├── routes/
│   │   ├── controllers/
│   │   └── models/
│   └── package.json
├── tests/
├── docs/
├── .gitignore
└── README.md
```

Figure 8.1: Proposed GitHub Repository Structure

The team can follow a simple version control workflow: obtain the latest code, implement an assigned task, test the changes, commit them with a meaningful message, push the commits to GitHub, and integrate the changes according to the team's agreed procedure. If branches and pull requests are used, changes can be reviewed before merging.

CHAPTER 9. TESTING & ACCEPTANCE

9.1 TESTING DOCUMENTATION

Testing in the 3D House Floor Plan Designer Project ensures that the application functions according to its requirements and provides users with a reliable environment for designing and visualizing residential floor plans. The application includes several interconnected modules, such as user authentication, project creation, 2D floor plan editing, room and wall management, doors and windows, dimension management, 3D visualization, furniture placement, project storage, sharing, and file export. Testing is required to verify the correctness of individual features and the integration between these modules.

TESTING PROCESS:

1. Unit Testing

Unit testing verifies individual functions and components independently. In the 3D House Floor Plan Designer, this includes testing room creation, wall placement, dimension calculations, unit conversion, input validation, project data handling, and geometry-related utility functions. Individual functions should be tested with valid, invalid, and boundary-value inputs to confirm that they produce the expected results.

2. Integration Testing

Integration testing verifies that different modules communicate and operate correctly together. For example, when a user creates a room in the 2D editor, the room geometry should be reflected correctly in the 3D visualization. Similarly, saving a floor plan should store the required project data, and reopening the project should restore the saved rooms, walls, openings, dimensions, and furniture where supported.

3. Functional Testing

Functional testing verifies complete user workflows against the system requirements. Typical workflows include registering or logging in, creating a project, designing a floor plan, adding doors and windows, viewing the design in 3D, saving the project, sharing it with permitted users, and exporting the design. The expected result of each operation should be compared with the actual application behavior.

4. Validation and Error-Handling Testing

This testing checks whether the application handles invalid or incomplete inputs appropriately. Examples include attempting to create a room with invalid dimensions, entering unsupported values, opening a nonexistent project, submitting incomplete login details, or importing an invalid blueprint file. The application should provide appropriate feedback and avoid saving inconsistent project data.

5. Usability and Compatibility Testing

Usability testing evaluates whether users can understand the editor controls, create floor plans, switch between 2D and 3D views, and perform common operations without unnecessary difficulty. Compatibility testing checks the application on supported browsers and screen sizes. The interface should remain usable on different devices, subject to the project's implemented responsive-design capabilities.

9.2 TEST PLANS DERIVED

Test plans for the 3D House Floor Plan Designer Project are derived from the acceptance criteria of each feature. Each test plan identifies the objective, inputs, expected outcome, and testing approach. These plans provide a systematic way to verify that the application's major modules operate correctly and satisfy the defined requirements.

1. User Authentication

- Objective: Verify that only users with valid credentials can access protected features.
- Inputs: Valid credentials, invalid credentials, and incomplete login details.
- Expected Outcome: Valid users can log in and log out successfully. Invalid or incomplete credentials should produce appropriate error messages.
- Testing Approach: Unit test input validation and authentication logic; integration test authentication with backend services and protected project operations.
- Acceptance Criteria: Authentication works as designed, errors are handled appropriately, and protected resources cannot be accessed without the required authorization.

2. Project Creation and Management

- Objective: Ensure that users can create and manage house design projects.
- Inputs: Project name, project details, and available project-management actions.
- Expected Outcome: A new project is created with the correct details and can be opened or modified using the implemented functionality.
- Testing Approach: Test project form validation, backend operations, database persistence, and project retrieval.
- Acceptance Criteria: Projects are created and managed correctly, and invalid project details are handled appropriately.

3. 2D Floor Plan Editor

- Objective: Verify that users can create and modify a two-dimensional house floor plan.
- Inputs: Room dimensions, wall positions, object selections, and editing operations.
- Expected Outcome: Rooms and walls appear at the intended positions, and supported editing operations update the floor plan correctly.
- Testing Approach: Unit test geometry calculations and validation utilities; perform functional tests on room creation, wall editing, selection, movement, and deletion.
- Acceptance Criteria: The editor represents the entered geometry correctly and prevents or reports invalid configurations according to the implemented rules.

4. Room and Wall Management

- Objective: Confirm that room and wall operations maintain a consistent floor plan.
- Inputs: Room dimensions, wall coordinates, and room or wall modification requests.
- Expected Outcome: Room boundaries and wall positions are updated correctly, and related geometry remains consistent.
- Testing Approach: Test geometry utilities independently and verify room-wall interactions through integration testing.

- Acceptance Criteria: Supported room and wall operations produce the expected layout without unintended geometry changes.

5. Doors, Windows, and Stairs

- Objective: Verify the placement and management of architectural openings and stairs.
- Inputs: Door, window, or stair selection; position; and available dimensions.
- Expected Outcome: Selected elements appear in the intended locations and are represented correctly in the floor plan and 3D view where supported.
- Testing Approach: Test element placement, positioning, dimension validation, and synchronization between the editor and visualization.
- Acceptance Criteria: Elements are displayed and edited correctly within the limits of the implemented design rules.

6. 3D Visualization

- Objective: Verify that the 3D model accurately represents the designed floor plan.
- Inputs: Floor plans containing rooms, walls, doors, windows, and other supported elements.
- Expected Outcome: The 3D view displays the corresponding spatial arrangement and reflects supported design changes.
- Testing Approach: Test Three.js rendering, geometry generation, object positioning, and integration with the 2D editor.
- Acceptance Criteria: The 3D representation corresponds to the underlying floor plan, and supported changes appear correctly without rendering errors.

7. Furniture Placement and Appearance

- Objective: Check furniture arrangement and available visual customization features.
- Inputs: Furniture items, placement coordinates, and supported appearance settings.
- Expected Outcome: Furniture and appearance changes are displayed in the correct locations and are retained when saving and reopening a project, where persistence is supported.
- Testing Approach: Test object placement, movement, deletion, rendering, and persistence.
- Acceptance Criteria: Supported objects and appearance settings behave as expected and do not unintentionally alter unrelated design elements.

8. Blueprint Import

- Objective: Verify that the application handles supported blueprint files correctly.
- Inputs: A valid supported blueprint file, an invalid file, and an unsupported file format.
- Expected Outcome: Valid supported files are processed according to the implemented import functionality. Invalid or unsupported files should be rejected or handled with an informative message.
- Testing Approach: Test file validation, import processing, error handling, and the resulting design data.
- Acceptance Criteria: The application processes supported input correctly and handles invalid input without crashing or corrupting existing project data.

9. Save and Load Project

- Objective: Ensure that users can save a floor plan and restore it later.
- Inputs: A project containing rooms, walls, openings, dimensions, and furniture where supported.
- Expected Outcome: The saved project can be reopened with its supported design elements and properties preserved.
- Testing Approach: Test serialization, backend requests, database storage, retrieval, and restoration of project data.
- Acceptance Criteria: Saved data is retrieved consistently, and supported project elements are not unintentionally lost or modified.

10. Project Sharing and Permissions

- Objective: Verify that projects can be shared with authorized users or viewers where the feature is implemented.
- Inputs: A valid project, sharing settings, and authorized or unauthorized access attempts.
- Expected Outcome: Intended recipients can access the shared project according to the configured permissions, while unauthorized access is restricted.
- Testing Approach: Test sharing operations, permission checks, shared-viewer access, and relevant backend authorization.
- Acceptance Criteria: Sharing works according to the defined access rules and does not expose projects to unauthorized users.

11. Validation and Error Handling

- Objective: Ensure that invalid operations are handled safely and clearly.
- Inputs: Invalid dimensions, incomplete forms, missing project identifiers, and malformed or unsupported files.
- Expected Outcome: The application displays suitable error feedback and prevents invalid data from being processed where required.
- Testing Approach: Perform negative testing, boundary-value testing, and unit tests for validation functions.
- Acceptance Criteria: Invalid input is handled consistently, errors do not cause unintended data loss, and the application remains usable after recoverable errors.

12. Testing and Deployment Setup

- Objective: Verify that the application and its required services can be started and used in the intended environment.
- Inputs: Application configuration, required environment variables, database connectivity, and deployment settings.
- Expected Outcome: The frontend, backend, and database operate together under the documented setup procedure.
- Testing Approach: Perform startup checks, API checks, database connectivity tests, and essential end-to-end workflows.
- Acceptance Criteria: The application starts correctly, required services are reachable, and core project-design operations work in the target environment.

9.3 RESULTS FROM TESTING

Testing of the 3D House Floor Plan Designer Project should be performed throughout development and after the integration of major features. The results should be evaluated against the acceptance criteria defined for each module. Unit testing verifies individual functions, integration testing checks communication between components, and functional testing validates complete user workflows.

Summary of Testing Results

1. User Authentication
    - Verified registration, login, and logout where implemented.
    - Checked the application's response to invalid credentials and incomplete input.
    - Verified access restrictions for protected operations where applicable.
    - Recorded the result of each test case as Pass or Fail.
2. Project Creation and Management
    - Checked whether a user could create a new floor plan project.
    - Verified that project details were saved and retrieved correctly.
    - Tested supported project-editing and deletion operations.
    - Checked for errors during project creation and retrieval.
3. 2D Floor Plan Editor
    - Tested room creation, wall placement, selection, movement, and deletion.
    - Verified that dimensions and coordinates were processed correctly.
    - Checked whether editing operations updated the floor plan as expected.
    - Recorded geometry-related defects and their resolution status.
4. Doors, Windows, Stairs, and Dimensions
    - Verified the placement of supported architectural elements.
    - Checked dimension calculations and unit conversions.
    - Confirmed that supported elements appeared correctly in the floor plan.
    - Compared expected and actual results for representative test inputs.
5. 3D Visualization
    - Checked whether rooms and walls were rendered correctly in the 3D view.
    - Verified the placement of supported doors, windows, stairs, and furniture.
    - Tested whether supported changes in the 2D editor were reflected in the 3D model.
    - Recorded any rendering, alignment, or geometry errors.
6. Furniture and Appearance
    - Tested the insertion, movement, and removal of furniture objects.
    - Checked supported materials, colours, and appearance settings.
    - Verified whether saved visual settings were restored when reopening a project, if supported.

7. Blueprint Import and Export
    - Tested valid and invalid blueprint inputs supported by the application.
    - Checked whether imported content was processed as expected.
    - Verified that supported PDF or image exports were generated and could be opened.
    - Recorded file-processing errors and output-quality issues.
8. Save, Load, and Sharing
    - Verified that project data was stored and restored correctly.
    - Checked whether supported design elements remained consistent after reopening.
    - Tested sharing links or viewer access where implemented.
    - Verified that access permissions were enforced according to the project's requirements.
9. Validation, Error Handling, and Deployment
    - Tested invalid dimensions, incomplete input, and unsupported operations.
    - Checked that suitable error messages were displayed.
    - Verified application startup, backend connectivity, and database access in the test environment.
    - Recorded any unresolved defects before final acceptance.

Test ID	Module	Expected Result	Actual Result / Status
TC01	Authentication	Valid login succeeds; invalid login is handled correctly	To be recorded
TC02	Project Creation	Project is created with the entered details	To be recorded
TC03	2D Editor	Rooms and walls are placed correctly	To be recorded
TC04	Architectural Elements	Doors, windows, and stairs behave as designed	To be recorded
TC05	Dimensions and Units	Measurements and conversions are correct	To be recorded
TC06	3D Visualization	3D model corresponds to the floor plan	To be recorded
TC07	Furniture Placement	Supported furniture operations work correctly	To be recorded
TC08	Blueprint Import	Supported files are processed correctly	To be recorded
TC09	Save and Load	Saved project data is restored correctly	To be recorded
TC10	Sharing and Permissions	Access follows configured permissions	To be recorded
TC11	Export	Supported output file is generated correctly	To be recorded
TC12	Validation	Invalid input is handled safely	To be recorded
TC13	Deployment	Application and required services start correctly	To be recorded

Table 9.1: Test Results Register

The final testing summary should state the number of test cases executed, passed, failed, and deferred. Any failed cases should be documented with the defect description, corrective action, and retest result. Tests for features that are not implemented should be marked Not Applicable or Not Implemented, rather than Pass.

9.4 USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) determines whether the implemented application meets the practical needs of its intended users. For the 3D House Floor Plan Designer Project, acceptance testing focuses on whether a user can create a residential floor plan, understand the layout, view it in three dimensions, and manage the design using the available features.

Acceptance Testing Process

1. Requirement Review: Identify the required features and their acceptance criteria.
2. Test Scenario Preparation: Prepare realistic scenarios, such as designing a simple house with a living room, bedroom, kitchen, doors, and windows.
3. Test Execution: Ask intended users or project evaluators to perform the supported tasks.
4. Result Verification: Compare the observed behaviour with the expected outcomes.
5. Defect Recording: Record issues relating to usability, design accuracy, data persistence, or feature behaviour.
6. Final Evaluation: Determine whether the implemented requirements have been satisfied and document any limitations.

User Acceptance Criteria

- Users can access the application and create a floor plan project using the available interface.
- Users can add and edit rooms and walls using supported editor operations.
- Doors, windows, stairs, and dimensions behave according to the implemented design rules.
- The 3D view represents the designed layout accurately enough for the project's stated purpose.
- Supported furniture and appearance features operate as expected.
- Project data can be saved and reopened without unintended loss of supported elements.
- Sharing and access control work according to the defined permissions, where implemented.
- Supported blueprint imports and export formats are processed correctly.
- Invalid input produces appropriate feedback and does not cause avoidable application failure.
- Essential workflows function in the target browser and deployment environment.

Acceptance Decision

The acceptance decision should be based on executed tests, requirement coverage, defect status, and evaluator feedback. Features that are incomplete or unavailable must be recorded as limitations. Acceptance should not be claimed solely because the application starts successfully; its core design workflows must also be evaluated.

CHAPTER 10. FINAL PRODUCT EVIDENCE

10.1 INTRODUCTION

Final Product Evidence presents the implemented features and overall functionality of the 3D House Floor Plan Designer Project. It provides visual and technical evidence that demonstrates how the application supports the creation, editing, visualization, storage, and management of residential floor plans. Screenshots of the application, test outputs, and relevant implementation details can be included to demonstrate the progress and completion of the project.

The evidence focuses on the user interface, 2D floor plan editor, architectural elements, 3D visualization, project management, and other implemented features. These records help project evaluators understand the system's functionality and verify its compliance with the defined requirements.

10.2 USER INTERFACE EVIDENCE

The user interface provides access to the major functions of the application. Screenshots of the home page, dashboard, navigation controls, and design workspace should be included to demonstrate the application's layout and usability.

Evidence to include:

- Application home page or dashboard.
- Project creation interface.
- Main floor plan design workspace.

10.3 2D FLOOR PLAN DESIGN EVIDENCE

The 2D floor plan editor enables users to design residential layouts by creating rooms, placing walls, and arranging supported architectural elements. Screenshots should demonstrate the editing interface and the arrangement of rooms and walls.

Evidence to include:

- Creation of rooms and walls.
- Editing and positioning of floor plan elements.
- Display of dimensions and measurements.
- Placement of doors, windows, and stairs where implemented.

10.4 3D VISUALIZATION EVIDENCE

The 3D visualization module presents the designed floor plan as a three-dimensional model. It helps users understand room arrangements, wall positions, and the spatial relationships between architectural elements. Screenshots should demonstrate the rendered model and, where supported, changes reflected after editing the 2D layout.

Evidence to include:

- Overall 3D house model.
- Three-dimensional view of rooms and walls.
- Furniture placement and appearance settings, where implemented.

10.5 PROJECT MANAGEMENT EVIDENCE

Project management functionality allows users to manage their house design projects. Evidence should demonstrate the available project creation, saving, loading, editing, and sharing operations.

Evidence to include:

- Creation of a new floor plan project.
- Saving and reopening a project.
- Project sharing and permission settings, where implemented.

Figure 10.4: Project Management and Saved Floor Plan.

10.6 IMPORT AND EXPORT EVIDENCE

The application may provide facilities for importing supported blueprint files and exporting completed floor plans into supported formats. Screenshots should demonstrate these operations when they are available in the implemented system.

Evidence to include:

- Blueprint import interface and successfully processed input.
- Export options for PDF or image formats.
- Example of an exported floor plan.

10.7 TESTING AND FUNCTIONALITY EVIDENCE

Testing evidence demonstrates whether the application's features operate according to their expected behaviour. Relevant screenshots or test reports should show successful workflows, validation messages, and the handling of invalid inputs.

Evidence to include:

- Results of floor plan creation and editing tests.
- Verification of 2D-to-3D visualization.
- Save-and-load verification.
- Validation and error-handling results.
- Application execution and deployment evidence, where applicable.

10.8 FINAL PRODUCT VERIFICATION

The final product should be evaluated against the implemented project requirements. The following checklist can be used to record the status of each major feature.

Feature	Evidence Required	Status
User interface	Home page or dashboard screenshot	To be verified
2D floor plan editor	Screenshot of rooms and walls	To be verified
Architectural elements	Doors, windows, and stairs	To be verified
3D visualization	Rendered house model	To be verified
Furniture placement	Furniture arrangement screenshot	To be verified
Project storage	Save-and-load test evidence	To be verified
Import and export	Sample supported file operations	To be verified
Sharing and permissions	Access verification, if implemented	To be verified
Testing	Actual test results	To be verified

Table 10.1: Final Product Verification Checklist.

APPENDICES

A.1 Frontend package configuration (excerpt)

```
{
 "name": "3d-house-designer",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "oxlint",
  "preview": "vite preview",
  "test": "vitest run",
  "test:watch": "vitest",
  "test:ui": "vitest --ui"
 },
 "dependencies": {
  "@types/canvas-confetti": "^1.9.0",
  "@types/three": "^0.185.4",
  "canvas-confetti": "^1.9.4",
  "html2canvas": "^1.4.1",
  "jspdf": "^4.2.1",
  "lucide-react": "^1.31.0",
  "react": "^19.2.8",
  "react-dom": "^19.2.8",
  "socket.io-client": "^4.8.3",
  "three": "^0.185.1"
 },
 "devDependencies": {
  "@tailwindcss/vite": "^4.3.3",
  "@types/node": "^24.13.3",
  "@types/react": "^19.2.17",
  "@types/react-dom": "^19.2.3",
  "@vitejs/plugin-react": "^6.0.4",
  "@vitest/ui": "^5.0.5",
  "jsdom": "^27.0.1",
  "oxlint": "^1.75.0",
  "tailwindcss": "^4.3.3",
  "typescript": "~6.0.2",
  "vite": "^8.2.0",
  "vitest": "^4.1.12"
 }
}
```

A.2 Backend server initialization (excerpt)

```
import http from 'http';
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import { Server as SocketIOServer } from 'socket.io';
import { connectDB } from './config/db.js';
import { seedDatabaseIfEmpty } from './services/dataStore.js';
import { initSocket } from './socket/socketHandler.js';
import authRoutes from './routes/authRoutes.js';
import professionalRoutes from './routes/professionalRoutes.js';
import chatRequestRoutes from './routes/chatRequestRoutes.js';
import conversationRoutes from './routes/conversationRoutes.js';
import messageRoutes from './routes/messageRoutes.js';
import projectRoutes from './routes/projectRoutes.js';
import verificationRoutes from './routes/verificationRoutes.js';
import paymentRoutes from './routes/paymentRoutes.js';
import adminRoutes from './routes/adminRoutes.js';
dotenv.config();
const app = express();
const server = http.createServer(app);
const PORT = process.env.PORT || 5000;
const CLIENT_URL = process.env.CLIENT_URL || 'http://localhost:5173';
// CORS Middleware
app.use(
  cors({
    origin: [CLIENT_URL, 'http://localhost:5173', 'http://127.0.0.1:5173', 'http://localhost:3000'],
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']
  })
);
app.use(express.json({ limit: '25mb' }));
app.use(express.urlencoded({ extended: true, limit: '25mb' }));
// Request Logging
app.use((req, res, next) => {
  if (req.path !== '/api/health') {
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  }
  next();
});
// Socket.IO Setup
const io = new SocketIOServer(server, {
  cors: {
    origin: [CLIENT_URL, 'http://localhost:5173', 'http://127.0.0.1:5173', 'http://localhost:3000'],
    credentials: true
  }
});
```

```
// Mount Routes
app.use('/api/auth', authRoutes);
```

A.3 Project data model (excerpt)

```
import mongoose from 'mongoose';
const projectSchema = new mongoose.Schema(
  {
    projectId: { type: String, required: true, unique: true },
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    name: { type: String, required: true },
    houseType: { type: String, default: 'Villa' },
    plotWidth: { type: Number, default: 40 },
    plotLength: { type: Number, default: 50 },
    unit: { type: String, enum: ['feet', 'meters'], default: 'feet' },
    floorsCount: { type: Number, default: 1 },
    thumbnail: { type: String, default: '' },
    walls: { type: Array, default: [] },
    rooms: { type: Array, default: [] },
    doors: { type: Array, default: [] },
    windows: { type: Array, default: [] },
    furniture: { type: Array, default: [] },
    isPublic: { type: Boolean, default: false }
  },
  { timestamps: true }
);
export const ProjectModel = mongoose.models.Project || mongoose.model('Project', projectSchema);
export default ProjectModel;
```

A.4 Project API route (excerpt)

```
import express from 'express';
import { db } from '../services/dataStore.js';
import { authenticateJWT } from '../middleware/auth.js';
const router = express.Router();
// GET /api/projects/:id - Retrieve project data for viewing/inspection
router.get('/:id', async (req, res) => {
  try {
    const project = await db.getProjectById(req.params.id);
    if (!project) {
      return res.status(404).json({ error: 'Project not found.' });
    }
    res.json({ project });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
// POST /api/projects - Save/sync a project to server
router.post('/', authenticateJWT, async (req, res) => {
```

```
  try {
    const projectData = req.body;
    if (!projectData || !projectData.name) {
      return res.status(400).json({ error: 'Project data is required.' });
    }
    const saved = await db.saveProject(projectData, req.user.id);
    res.json({ project: saved });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
export default router;
```

A.5 PDF report generator (excerpt)

```
import jsPDF from 'jspdf';
import { Project } from '../../types/floorplan';
import { formatArea } from '../../utils/unitConverter';
export const generatePdfReport = async (project: Project, viewMode?: string): Promise<void> => {
  const doc = new jsPDF({
    orientation: 'portrait',
    unit: 'mm',
    format: 'a4'
  });
  const pageWidth = doc.internal.pageSize.getWidth();
  const unit = project.unit;
  // Header Banner
  doc.setFillColor(15, 23, 42); // Deep Slate
  doc.rect(0, 0, pageWidth, 40, 'F');
  doc.setTextColor(56, 189, 248); // Cyan Accent
  doc.setFontSize(22);
  doc.setFont('helvetica', 'bold');
  doc.text('3D HOUSE FLOOR PLAN SPECIFICATION', 14, 18);
  doc.setTextColor(248, 250, 252);
  doc.setFontSize(12);
  doc.setFont('helvetica', 'normal');
  doc.text(`Project: ${project.name}`, 14, 28);
  doc.text(`Generated: ${new Date().toLocaleDateString()}`, pageWidth - 60, 28);
  // Metadata Box
  doc.setFillColor(241, 245, 249);
  doc.rect(14, 48, pageWidth - 28, 28, 'F');
  doc.setDrawColor(203, 213, 225);
  doc.rect(14, 48, pageWidth - 28, 28, 'S');
  doc.setTextColor(15, 23, 42);
  doc.setFontSize(10);
  doc.setFont('helvetica', 'bold');
  doc.text('PROPERTY OVERVIEW', 18, 56);
  doc.setFont('helvetica', 'normal');
```

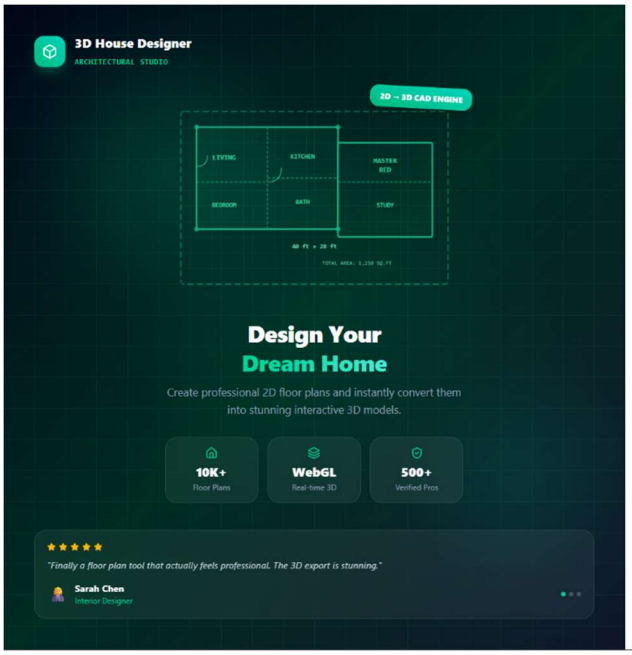
```
doc.text(`House Type: ${project.houseType}`, 18, 64);
doc.text(`Plot Width: ${project.plotWidth} ${unit}`, 18, 70);
doc.text(`Plot Length: ${project.plotLength} ${unit}`, 80, 64);
doc.text(`Floors: ${project.floorsCount} Level(s)`, 80, 70);
const totalArea = project.rooms.reduce((acc, r) => acc + r.area, 0);
doc.setFont('helvetica', 'bold');
doc.text(`Total Interior Area: ${formatArea(totalArea, unit)}`, 140, 64);
// Room Breakdown Table
doc.setFontSize(12);
doc.setFont('helvetica', 'bold');
doc.text('ROOM SPECIFICATIONS & AREA BREAKDOWN', 14, 88);
let yPos = 96;
doc.setFillColor(30, 41, 59);
doc.rect(14, yPos, pageWidth - 28, 8, 'F');
doc.setTextColor(255, 255, 255);
```

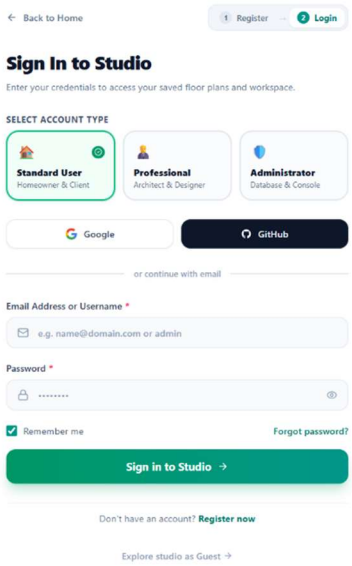
A.6 Source inventory

- src/App.tsx and src/main.tsx – frontend application entry and root composition.
- src/components/editor/canvas2d/ – 2D canvas, controls, scale calibration and validation panel.
- src/components/viewer3d/ – 3D canvas and object rendering helpers.
- src/components/editor/sidebar/ – build, floors, selection and property controls.
- src/components/upload/ and src/utils/detection/ – upload and blueprint analysis/validation modules.
- src/utils/geometry.ts, floorplanValidation.ts, unitConverter.ts – reusable domain utilities.
- src/components/export/PdfReportGenerator.ts – project specification PDF generation.
- server/src/routes/ – authentication, project, professional, chat, message, payment, verification and admin route modules.
- server/src/models/ – project, user, professional, conversation, message, notification, payment and verification data models.
- src/utils/__tests__/ – tests for floor-plan validation, geometry and unit conversion.

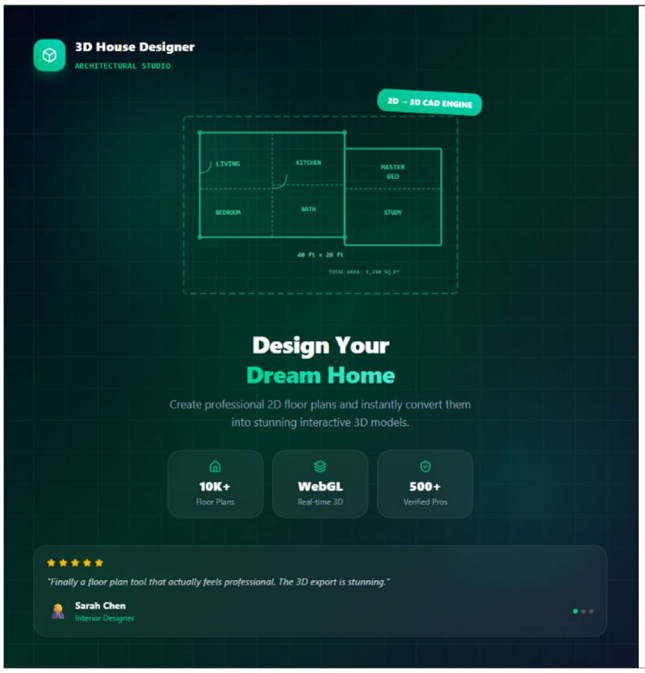
Before submission, remove node_modules and local secrets from any source archive shared externally. Retain package manifests and lockfiles so dependencies can be restored reproducibly.

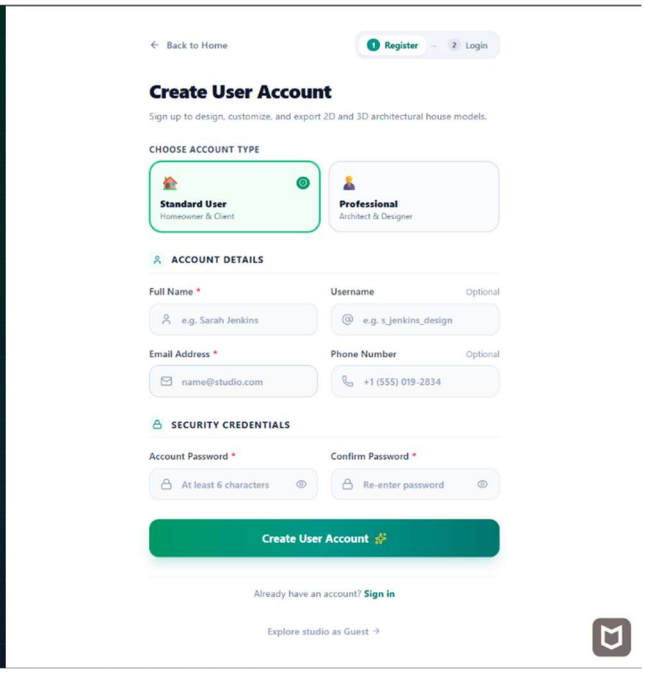
Login Page



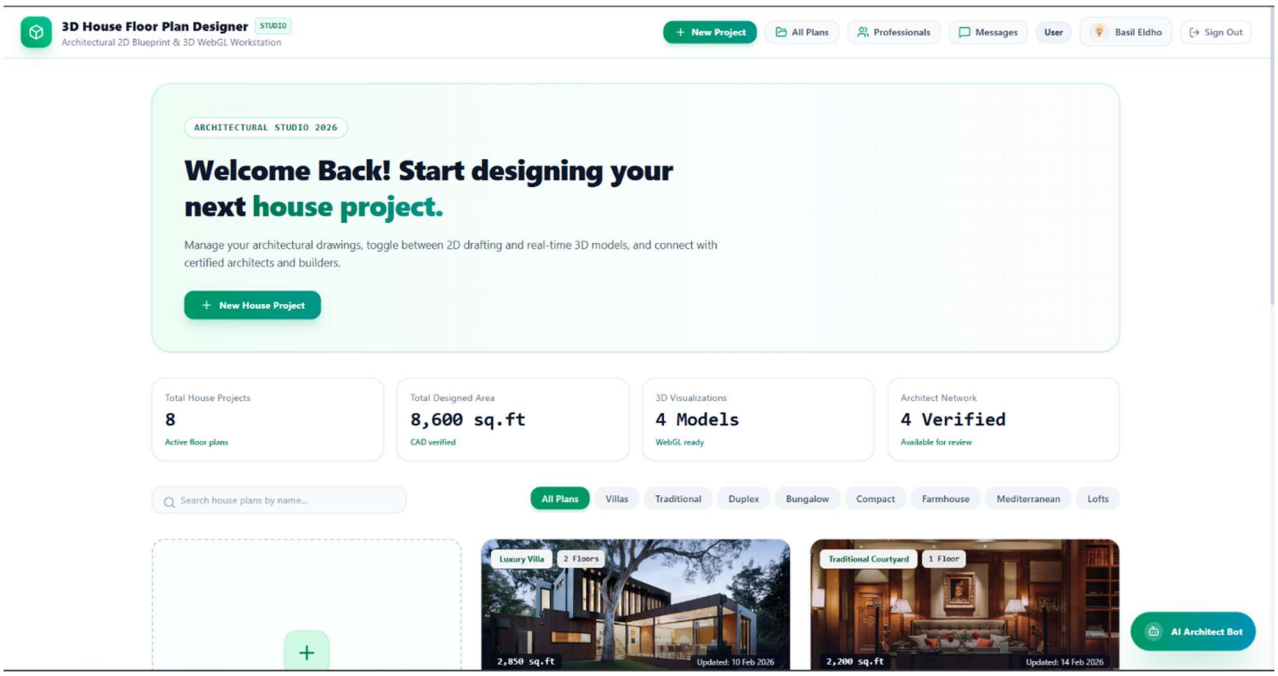


Register Page

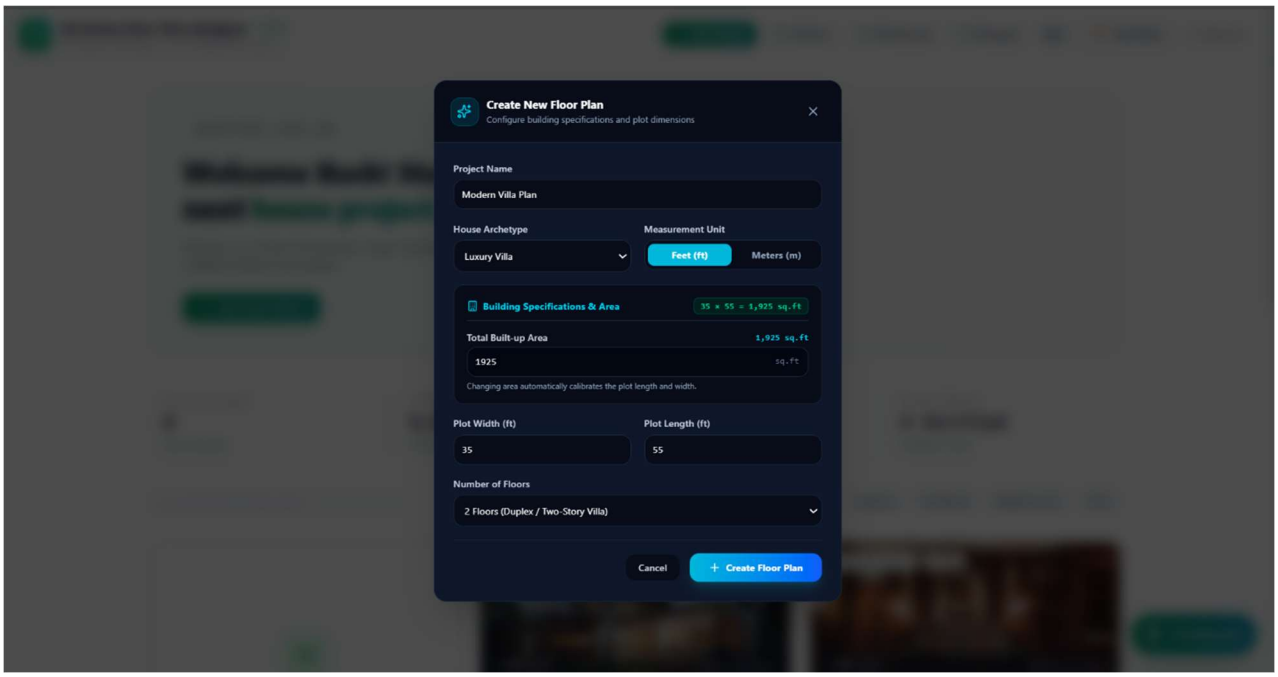




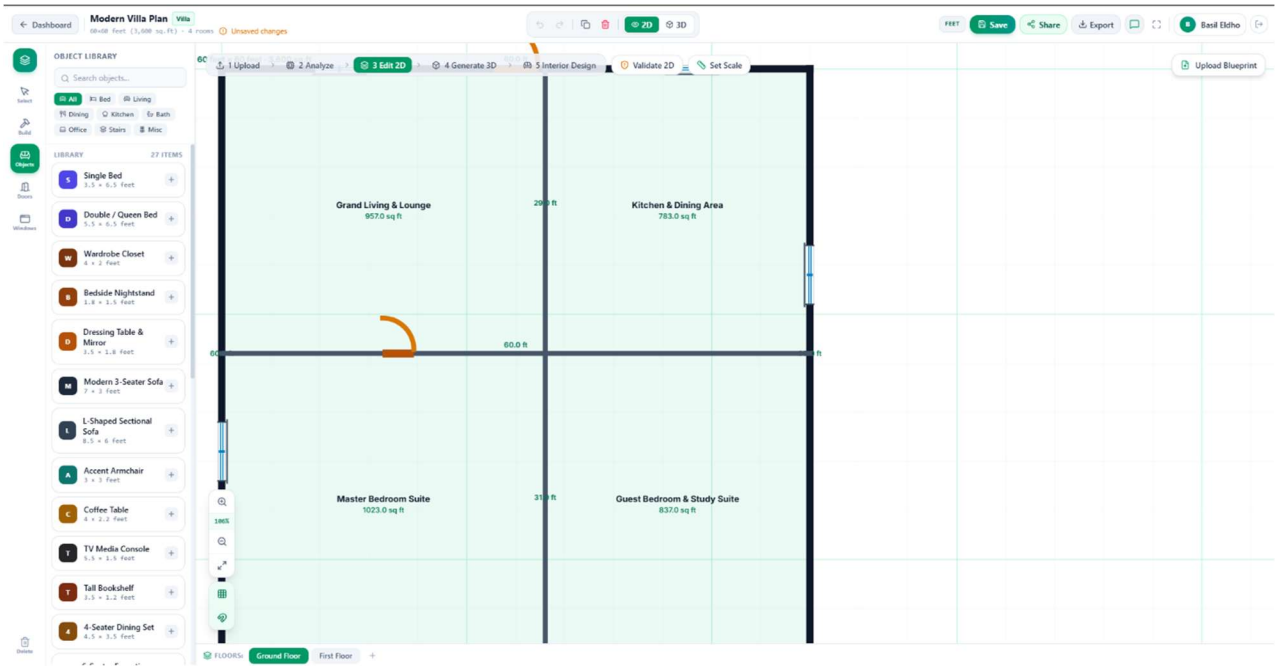
User Home Page



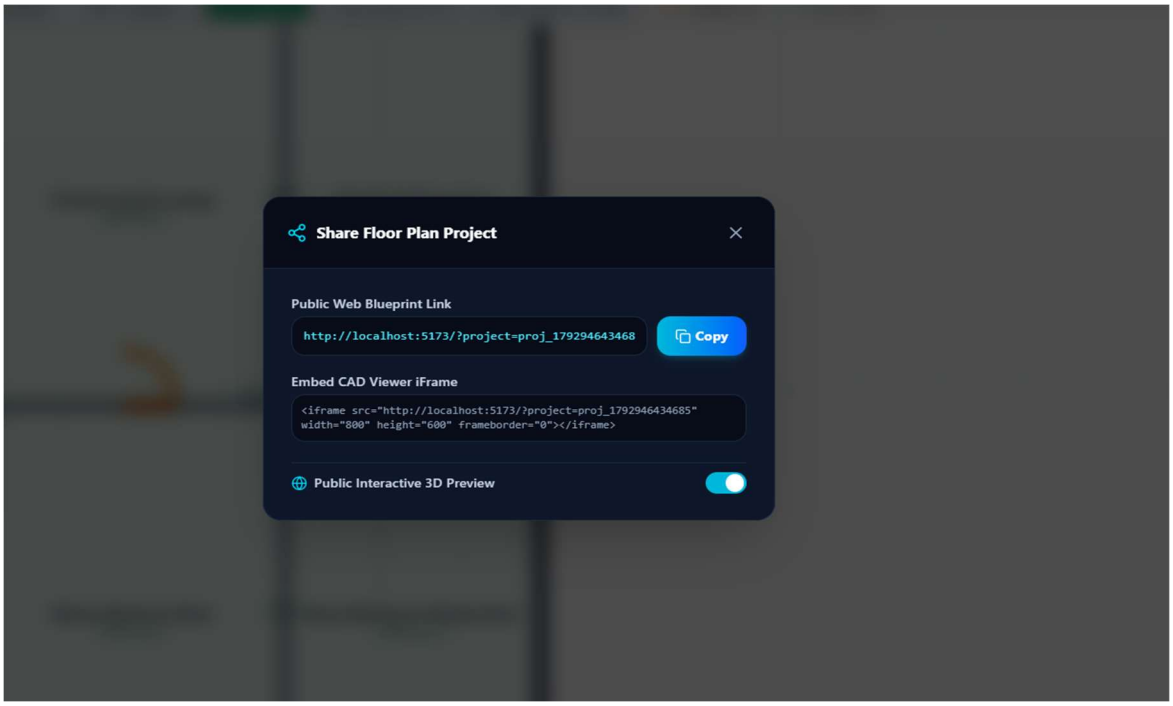
Dimensions



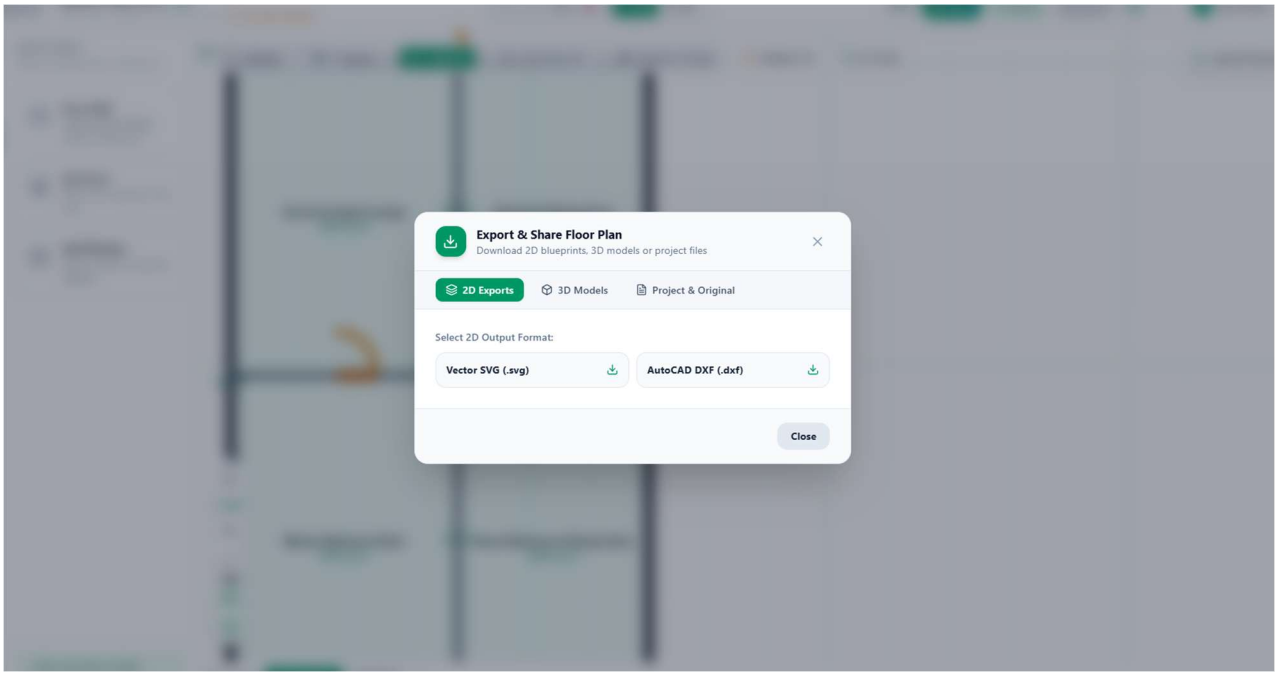
Workplace



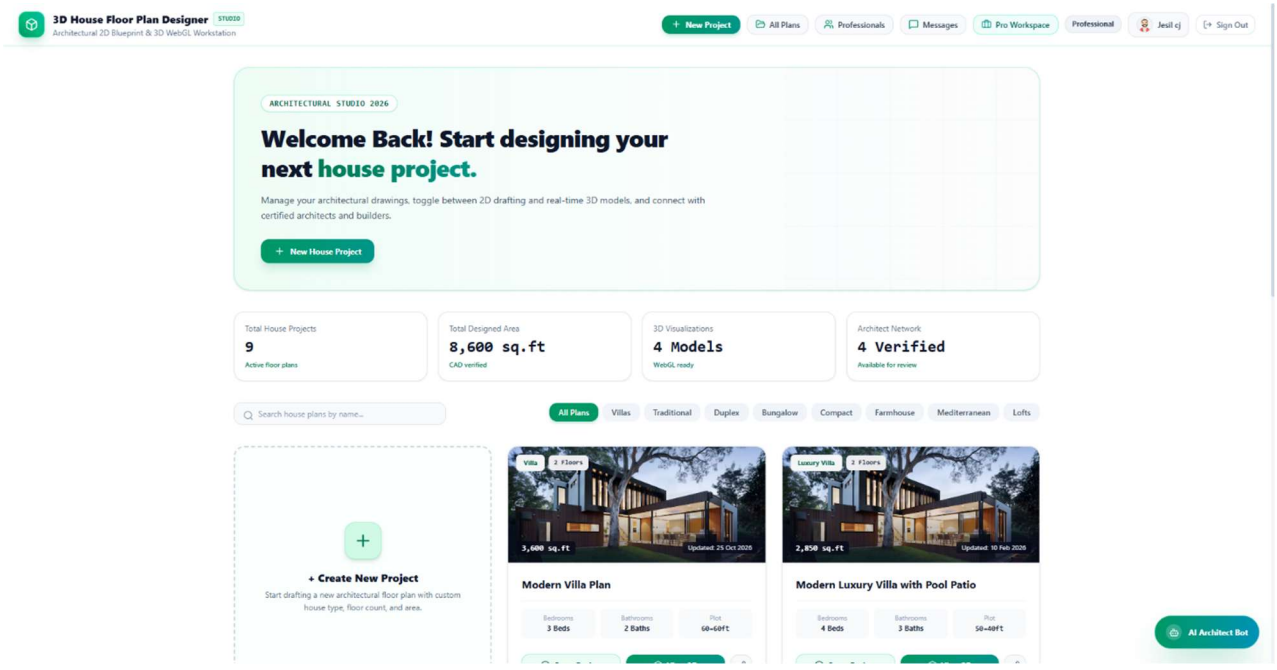
Plan Share



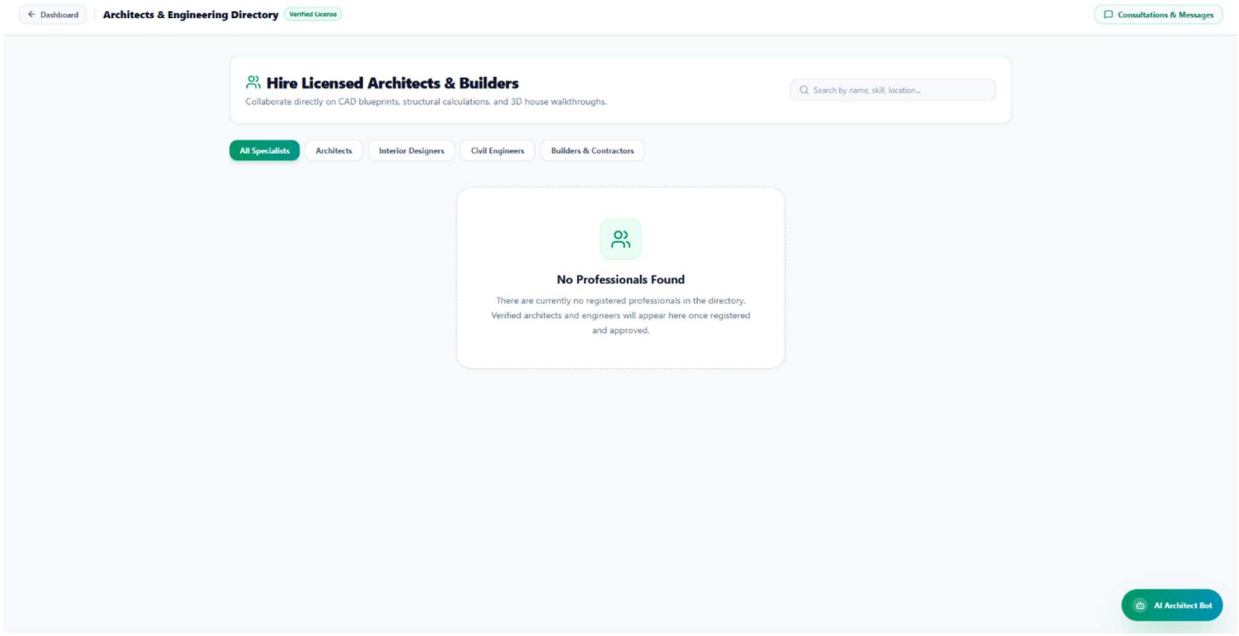
Download Plan



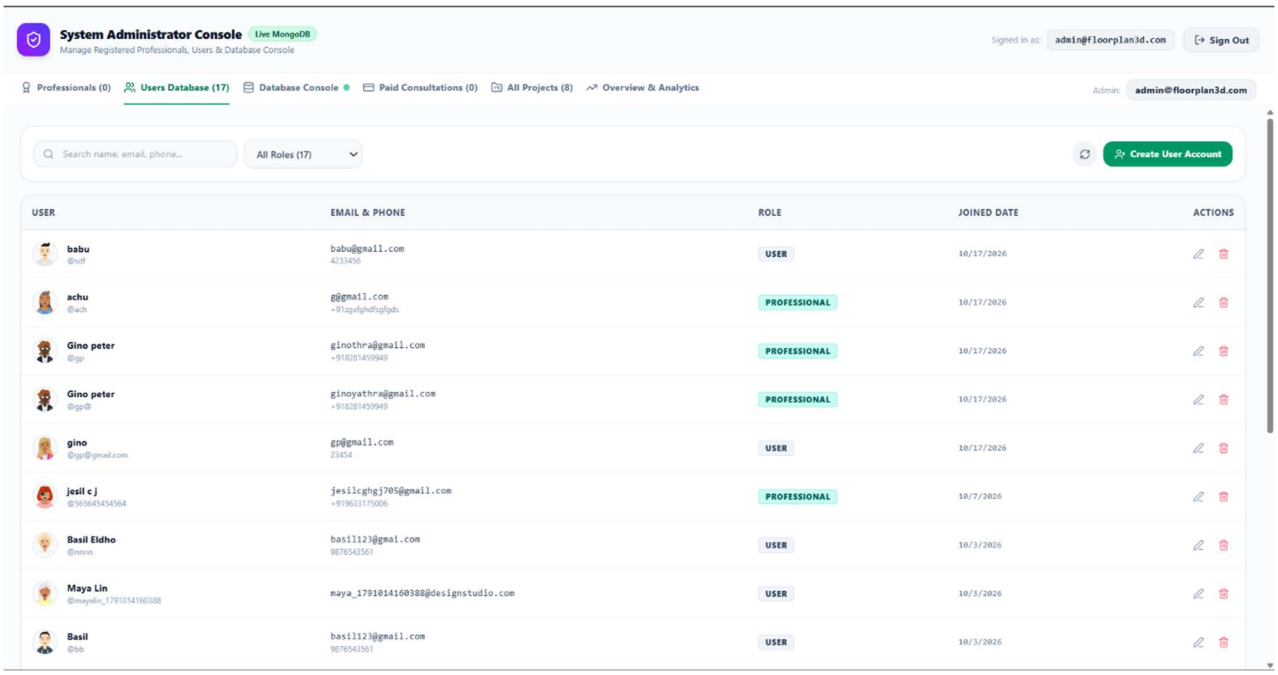
Professional Home Page



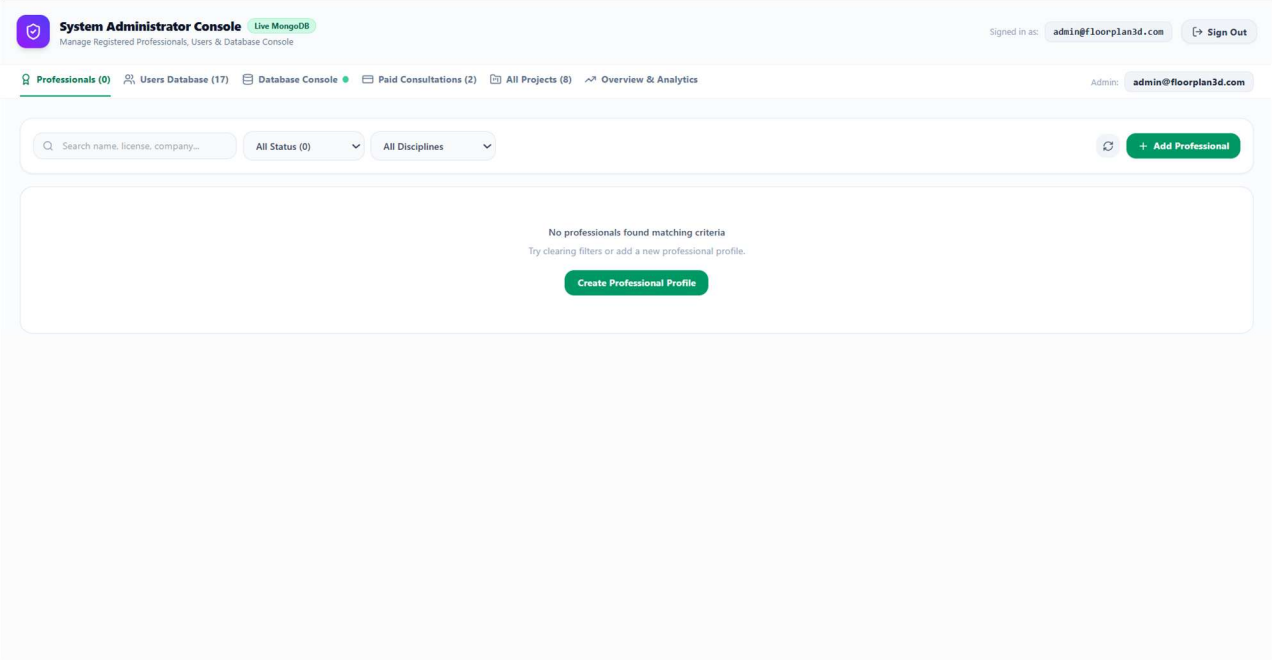
User connect



Admin Dashboard



Admin Dashboard



REFERENCES

1. React Documentation. https://react.dev/

Description: Official documentation for React, a JavaScript library for building component-based user interfaces, including components, state, and event handling.

2. Three.js Documentation. https://threejs.org/docs/

Description: Official documentation for Three.js, a JavaScript 3D library used to create and display browser-based 3D scenes, cameras, geometries, materials, and lighting.

3. MDN Web Docs — WebGL API. https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API

Description: Explains WebGL, a browser API for rendering interactive two-dimensional and three-dimensional graphics.

4. Node.js Documentation. https://nodejs.org/en/learn

Description: Official learning resources for Node.js, a JavaScript runtime used for server-side application logic.

5. Express.js Documentation. https://expressjs.com/

Description: Official documentation for Express.js, a Node.js framework for building web servers, routes, middleware, and APIs.

6. MongoDB Documentation. https://www.mongodb.com/docs/

Description: Official reference for MongoDB document storage, database queries, and data-management operations.

7. JSON Web Token (JWT) — RFC 7519. https://www.rfc-editor.org/rfc/rfc7519

Description: Internet Engineering Task Force specification describing the JSON Web Token format for transmitting claims between parties.

8. OWASP Authentication Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Description: Security recommendations for authentication implementation, login handling, and reducing common authentication risks.

9. The Scrum Guide. https://scrumguides.org/scrum-guide.html

Description: Official guide to Scrum theory, roles/accountabilities, events, artifacts, and commitments for iterative project management.

10. MDN Web Docs — HTML. https://developer.mozilla.org/en-US/docs/Web/HTML

Description: Reference for HTML, the markup language used to structure web pages and interface content.

11. MDN Web Docs — CSS. https://developer.mozilla.org/en-US/docs/Web/CSS

Description: Reference for CSS, which controls the presentation, layout, and responsive styling of web interfaces.

12. MDN Web Docs — JavaScript. https://developer.mozilla.org/en-US/docs/Web/JavaScript

Description: Reference for JavaScript language features used to implement application logic and interactive browser behaviour.

Note: These references provide technical background. The final report should describe only the features actually implemented and tested in the submitted application.